

PathPro

Technical documentation

See the risks on your way, before you go.

Live: pathpro.tech

Team CodingClaws, HackGT 13: Nagur Shareef Shaik, Sahith Reddy Thummala, Pranav Nagothu, Geethanjali Nagaboina

Model bundle: pp-20260926-1902-f49c0d2 · **Date:** September 26, 2026

Contents

1. [System architecture](#)
2. [Data and models](#)
3. [API reference](#)
4. [Deployment and operations](#)
5. [Security, privacy, and responsible AI](#)
6. [Testing and quality](#)

PathPro system architecture

PathPro forecasts where and when people on foot (and on bikes and scooters) are exposed to traffic crashes in Atlanta. It shows that forecast as an hourly map (Risk Tides) and uses it to plan lower-risk routes. This document describes how the running system is put together. It is written from the code in `backend/app`, `frontend/src`, `data/src/pathpulse_data`, and `deploy/`, as of bundle `pp-20260926-1902-f49c0d2`.

Fact	Value	Where it comes from
Live site	<code>https://pathpro.tech</code> (also <code>www.pathpro.tech</code> and <code>155-138-233-35.sslip.io</code>)	<code>deploy/go.sh</code> , <code>deploy/Caddyfile</code>
Host	One Vultr VM in Atlanta (<code>atl</code> , <code>vc2-1c-2gb</code> , Ubuntu 24.04), IP <code>155.138.233.35</code>	<code>deploy/provision_vultr.sh</code>
Model bundle served	<code>pp-20260926-1902-f49c0d2</code>	<code>artifacts/current</code> symlink, <code>/meta</code>
Street segments (risk units)	49,915 road segments across the City of Atlanta	<code>manifest.json</code> → <code>n_segments</code>
Walk routing graph	84,758 nodes, 120,763 edges (99,119 inherit a road segment)	<code>manifest.json</code> → <code>walk_graph</code>
Ride routing graph	49,824 nodes, 65,997 edges (53,158 inherit a road segment)	<code>manifest.json</code> → <code>ride_graph</code>
City Pulse grid	3,537 H3 resolution-9 hexes	<code>manifest.json</code> → <code>n_hexes</code>
API process	FastAPI under uvicorn, one worker, bound to <code>127.0.0.1:8000</code>	<code>deploy/pathpulse.service</code>

What the system produces (captured with Playwright from the offline demo, except City Pulse, which comes from the live site; see `docs/images/gallery/README.md`):

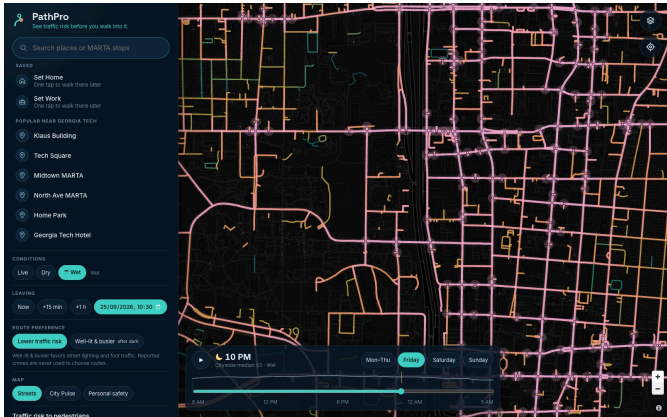
Phone and personal-safety views

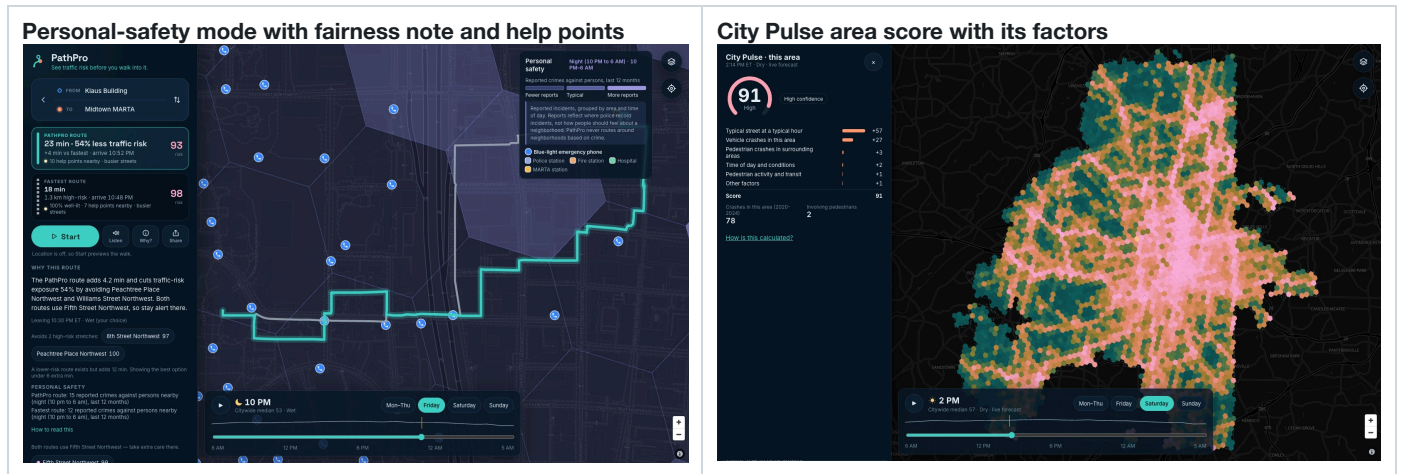
Fastest vs PathPro route (phone)



Desktop and City Pulse views

Risk Tides on desktop





Related documents: [data and models](#), [API reference](#), [deployment and operations](#), [security and privacy](#), [testing and quality](#).

1. Design principles that shape the architecture

- The model bundle is the hot path.** Every score, route, and explanation is computed from files loaded into memory at startup (`repositories/artifacts.py`). No database sits on the request path for routing or scoring (PRD NFR-04).
- Every external dependency is optional.** LLMs, voice, geocoding, weather, Tiger Data, and MongoDB Atlas each have a fallback. Missing keys turn a feature off; they never stop the app (section 9).
- The LLM never produces a number.** Scores and factor points come from the model; the LLM only rewrites server-built evidence, and a validator rejects anything not grounded in it (section 6.2).
- Crime never reaches routing or the traffic model.** The personal-safety layer passes only lighting and foot-traffic codes to the router (section 8).
- Versioned, immutable artifacts.** A bundle directory is named by model version, every file is hashed in `manifest.json`, and the browser fetches model files from `/static/<model_version>/` with a one-year immutable cache header.

2. Context (C4 level 1)

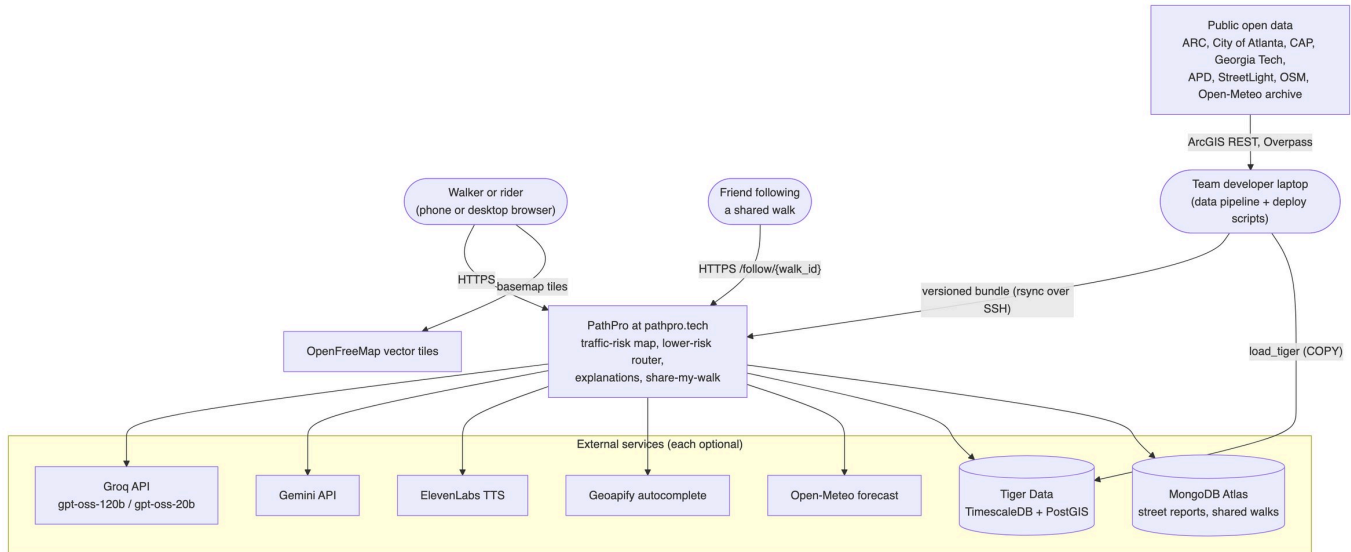


Figure A1. System context.

- **Users** reach one origin. The browser also loads basemap tiles directly from OpenFreeMap (allowed by the CSP `img-src/connect-src`).
- **The model is built offline.** The `data/` pipeline runs on a developer machine, pulls public layers with no accounts, and ships a bundle directory to the VM. The server never trains or fetches training data.
- **Tiger Data** is the system of record for crashes and the versioned score grid. The API reads it for exactly one feature: the "when crashes happened here" hourly chart.
- **MongoDB Atlas** stores user-generated state: community street reports and Share-my-walk sessions. Neither feeds the model.

3. Containers (C4 level 2)

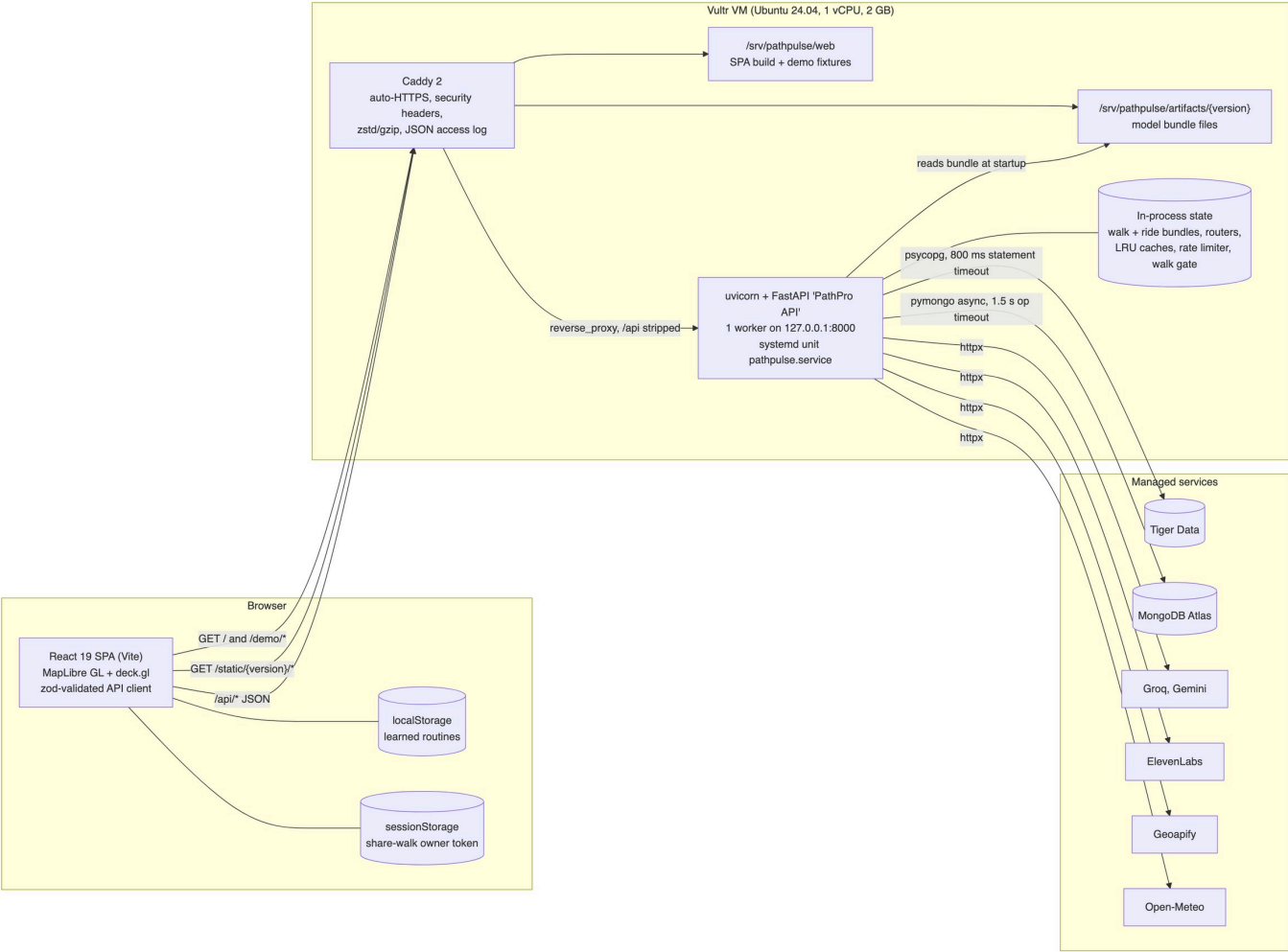


Figure A2. Containers.

Container	Technology	Responsibility	State
SPA	React 19, Vite 8, TypeScript 6, MapLibre GL 5, deck.gl 9, zod 4	Map, Risk Tides playback, search, route sheets, navigation, share, safety layers	URL query (view state), localStorage (routines), sessionStorage (share session)
Caddy	Caddy 2 (apt), config deploy/Caddyfile	TLS for three hostnames, security headers, static files, /api/* proxy	Certificates, JSON access log /var/log/caddy/pathpulse.log
API	FastAPI, uvicorn --workers 1 --proxy-headers --forwarded-allow-ips 127.0.0.1	Routing, scoring, attribution, explanations, voice, geocoding proxy, reports, shared walks, safety queries	In memory only (see section 4.3)
Bundle	Files under /srv/pathpulse/artifacts/<version>/, current symlink	Model outputs: factors, frames, graphs, geometry, metrics	Immutable per version; old versions kept

Tiger Data	TimescaleDB + PostGIS (managed)	<code>crashes</code> hypertable, <code>crashes_hourly</code> continuous aggregate, <code>segments</code> , <code>risk_grid</code>	System of record
Mongo DB Atlas	M0 cluster, collections <code>street_reports</code> , <code>shared_walks</code>	User-generated context and live sharing	TTL-expiring documents

4. Backend internals

4.1 Layers

Package	Role	Examples
<code>app/api/</code>	FastAPI routers, request/response schemas, the response envelope, dependencies	<code>core.py</code> (<code>/healthz</code> , <code>/meta</code> , <code>/routes</code> , <code>/segments/{id}</code>), <code>extras.py</code> , <code>areas.py</code> , <code>reports.py</code> , <code>walks.py</code> , <code>safety.py</code> , <code>transit.py</code> , <code>envelope.py</code>
<code>app/services/</code>	Use cases that combine domain logic with I/O	<code>routing.py</code> (coverage, snapping, copy), <code>segments.py</code> , <code>explain/*</code> , <code>weather.py</code> , <code>geocode.py</code> , <code>tts.py</code> , <code>reports.py</code> , <code>walks.py</code> , <code>safety.py</code>
<code>app/domain/</code>	Pure logic, no I/O	<code>router.py</code> (Dijkstra ladder), <code>route_metrics.py</code> , <code>scoring.py</code> (percentile scale, attribution), <code>alerts.py</code> , <code>timeutil.py</code> , <code>modes.py</code> , <code>safety.py</code> , <code>walks.py</code> , <code>reports.py</code> , <code>transit.py</code>
<code>app/repositories/</code>	Data access with tight timeouts	<code>artifacts.py</code> (bundle loader), <code>hexes.py</code> , <code>safety.py</code> , <code>history.py</code> (Tiger), <code>reports.py</code> and <code>walks.py</code> (Atlas)
<code>app/tools/</code>	Operator CLIs, not served	<code>check_keys.py</code> (verifies each sponsor key without printing it), <code>record_demo.py</code> (records the offline demo)

4.2 Startup (create_app in app/main.py)

- `load_bundle(ARTIFACTS_DIR)` loads the walk model and **fails fast**: a missing file or a SHA-256 mismatch against `manifest.json` raises `BundleError`, and the process does not start.
- `load_safety(...)` loads the personal-safety layer if all five safety files exist and their shapes match the walk graph; otherwise `None`.
- `Router(bundle, safety.edges)` builds the walk router (directed edge arrays, a k-d tree for snapping, parallel-edge index).
- `load_ride_bundle(root)` loads the `ride_`-prefixed model if `ride_graph.npz` exists; any error disables ride modes and logs a warning. A second `Router` is built for it.
- `load_hexes(root)` loads City Pulse if `hex_meta.json` exists.
- In-memory objects: a 512-entry LRU of recent `RoutesData` (evidence for `/explain`), the rate limiter, and the Share-my-walk update gate.
- Repositories for Tiger Data and Atlas are constructed **without I/O**. The lifespan hook creates one shared `httpx.AsyncClient`, the explanation, geocoding, and TTS services, and asks Atlas to ensure indexes (failure only logs a warning).
- Middleware: `CORSMiddleware` (added last, so outermost) wraps `RateLimitMiddleware`. Exception handlers render every error as the standard envelope.
- The bundle directory is also mounted at `/static/<model_version>` by FastAPI (Caddy serves the same path from disk in production).

4.3 In-process state and the single worker

Several features rely on memory that is local to one process:

State	Where	Consequence
<code>routes_cache</code> (LRU 512)	<code>app.state.routes_cache</code>	<code>POST /explain {kind: "route"}</code> finds the route by <code>route_key</code> . A different process, or a restart, returns <code>ROUTE_EXPIRED</code> (404) and the client re-requests the route.
Explanation cache (LRU 4,096) and single-flight locks	<code>ExplainService</code>	One LLM call per unique evidence key, per process.
Rate-limit windows (TTL cache, 50,000 clients)	<code>RateLimitMiddleware</code>	Limits are per process.
Share-my-walk update gate (TTL cache, 10,000 walks)	<code>RecentUpdates</code>	Floods are rejected before any Atlas read.
Daily budgets for LLM, geocoding, TTS	<code>DailyBudget</code>	Counters reset at local midnight and on restart.

The systemd unit runs `--workers 1` so these behave as designed. At about 120 ms of CPU per route (the router runs in `asyncio.to_thread` to keep the event loop free), one worker fits a hackathon load on a 1-vCPU VM. Scaling out would need a shared store for the route cache and limiter.

4.4 Frontend structure

Area	Files	Notes
Shell	<code>main.tsx</code> , <code>App.tsx</code> , <code>app/PathPro.tsx</code>	<code>initRuntime()</code> resolves the API origin before first render (same-origin on the VM). <code>PathPro</code> owns view state and wires the data hooks.
API client	<code>api/client.ts</code> , <code>api/schemas.ts</code> , <code>api/safetySchemas.ts</code> , <code>api/walks.ts</code> , <code>api/safety.ts</code>	Every response is parsed with a zod schema wrapped in the envelope schema; an 8 s abort timeout; non-JSON bodies become a friendly <code>SERVER</code> error.
Demo transport	<code>api/demo.ts</code>	With <code>?demo=1</code> , <code>request()</code> answers from <code>/demo/fixtures.json</code> instead of the network (section 6.4).
Bundle loading	<code>hooks/useBundle.ts</code> , <code>hooks/bundleFiles.ts</code> , <code>hooks/useRideNetwork.ts</code>	Loads <code>/meta</code> , then <code>segments.geojson</code> , <code>hotspot_nodes.json</code> , <code>hex_cells.json</code> from <code>meta.static_base</code> . The ride geometry and frames load only when a ride mode is first chosen.
Risk Tides frames	<code>frames/frameStore.ts</code> , <code>hooks/useRiskFrames.ts</code>	One <code>Uint8Array</code> per (day group, condition); <code>hour(h)</code> is a zero-copy subarray, so scrubbing never touches the network after the first fetch (PRD NFR-02).
Map	<code>map/MapView.tsx</code> , <code>map/layers.ts</code> , <code>map/hexLayer.ts</code> , <code>map/safetyLayers.ts</code> , <code>map/FollowMap.tsx</code>	MapLibre basemap with <code>deck.gl PathLayer</code> , <code>ScatterplotLayer</code> , <code>SolidPolygonLayer</code> , and H3 hex layers.
On-device features	<code>lib/routines.ts</code> , <code>lib/shareSession.ts</code> , <code>hooks/useNavigation.ts</code> , <code>hooks/useGeolocation.ts</code>	Routines are learned in <code>localStorage</code> only; navigation follows the GPS watch and speaks alerts; nothing about walk history is sent to the server.

View state	state/urlState.ts	Origin, destination, departure, condition, hour, day, segment, preference, and mode live in the URL so a view survives refresh.
------------	-------------------	---

5. How a score is computed at request time

Every score in the product comes from one identity that the pipeline guarantees (checked to within 1e-6 at export):

```

log density(segment, cell) = base + Σ spatial factor(segment) + Σ temporal factor(cell)
score = percentile of that log density on the bundle's 1,001-point citywide quantile table (0–100)
cell = (day group, hour, light, wet) # Atlanta local time; light from solar elevation

```

- `Bundle.log_density()` is one vector addition, so the router can score all 240k directed edges for a departure in a single NumPy pass.
- `domain/scoring.attribute()` splits a displayed score into a baseline, the top five factors, and a remainder, by adding factors from largest to smallest effect and assigning each the score change it causes. The integers always sum to the score (property-tested on 200 random cases in `backend/tests/test_scoring.py`).
- Bands: Lower 0–24, Moderate 25–49, Elevated 50–74, High 75–100.

Details of how the factors are fitted are in [data_and_models.md](#).

6. Request flows

6.1 Plan a route

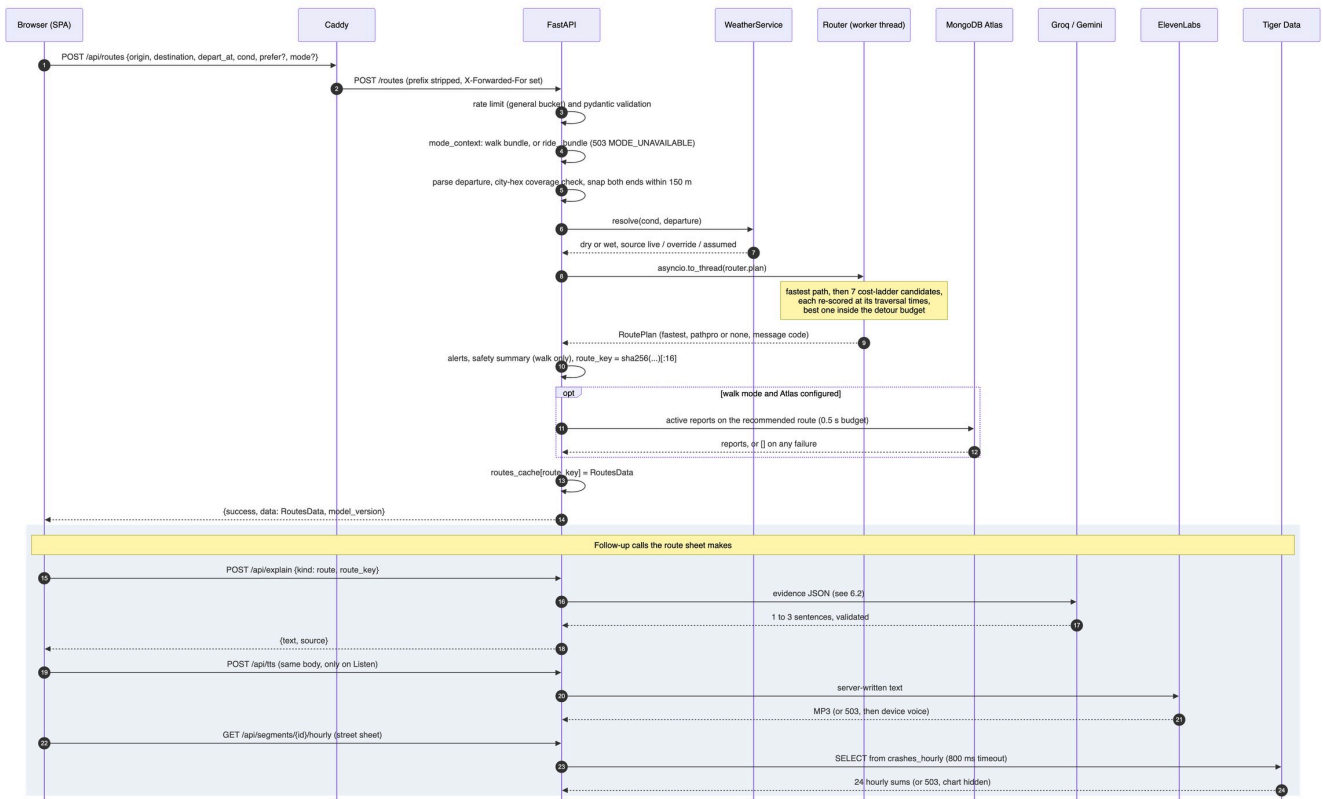


Figure A3. Plan-a-route sequence.

What the router does (domain/router.py, domain/route_metrics.py):

Step	Rule
Too close	Origin and destination on the same node or under 60 m apart → T00_CLOSE
Search area	Directed edges inside a box around the trip padded by $\max(1,200 \text{ m}, 0.6 \times \text{trip distance})$; if the box cuts the only connection, the whole graph is searched
Fastest path	Dijkstra (SciPy <code>csgraph</code>) on travel time at the mode's speed (walk 1.3 m/s; bike 15, e-bike 22, scooter 18 km/h)
Candidates	Dijkstra with cost = $\text{time} \times (1 + \lambda \times \text{density} / \text{density at score 75})$ for $\lambda \in \{0.05, 0.1, 0.25, 0.5, 1, 2, 4\}$
Re-scoring	Each candidate is re-scored edge by edge at the minute the walker would reach that edge (light and hour can change mid-trip)
Detour budget	$\min(1.25 \times \text{fastest duration}, \text{fastest} + 6 \text{ min})$
Pick	Lowest expected-crash exposure inside the budget; shown only if it is ≥ 10 score points lower or has $\geq 15\%$ less exposure, else "The fastest route is already the lower-risk option."
Trade-off note	If a candidate outside the budget cuts exposure a further $\geq 15\%$, the message says how many minutes it would add
Long trips	Walk: $> 5 \text{ km}$ or $> 1 \text{ h}$; ride: $> 15 \text{ km}$ or $> 1 \text{ h}$ → <code>long_trip</code> message suggesting MARTA
Lit and busy	Walk only, after dark only, opt-in: see section 8

6.2 "Why?" explanations

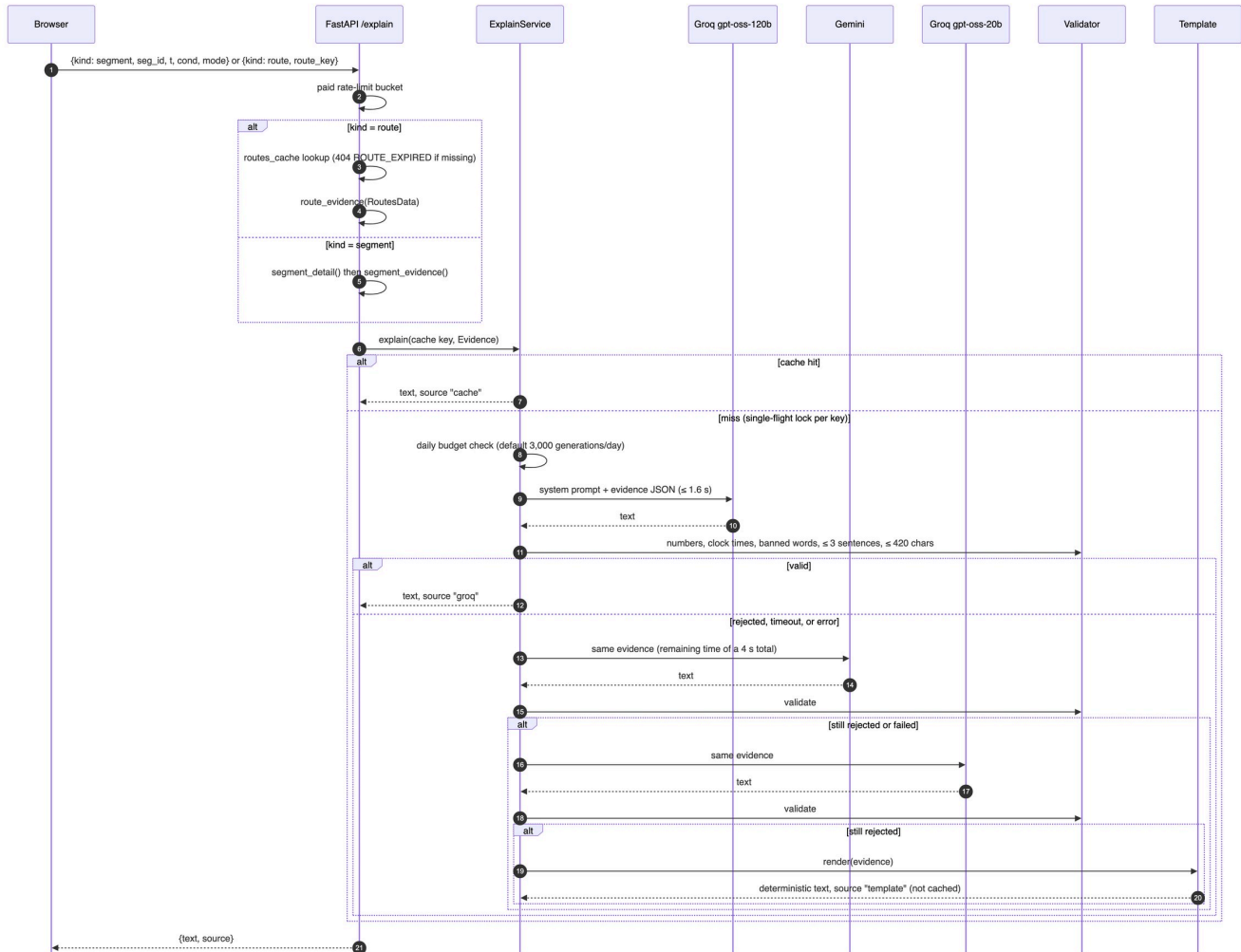


Figure A4. Explanation chain.

- **Evidence is the only LLM input.** `services/explain/evidence.py` builds a JSON payload from server data (street name, score, band, time label, conditions, confidence, up to three factors that raise risk and one that lowers it, crash history; or, for routes, minutes, scores, extra minutes, exposure reduction, and avoided streets). No user text is included.
- **Cache keys** include the model version: `seg:{id}:{day|hour|light|wet}:{version}[:{mode}]` and `route:{route_key}:{version}`.
- **Validator** (`services/explain/validator.py`) rejects text that is empty, longer than 420 characters, longer than three sentences, contains a banned framing, contains a clock time other than the evidence time, or contains any number not present in the evidence (numbers inside evidence strings such as street names or "2020-2024" count as grounded).
- **Provider order** is built in `main.build_providers`: Groq (GROQ_MODEL, default `openai/gpt-oss-120b`), Gemini (GEMINI_MODEL, default `gemini-3.8-flash`), then Groq (GROQ_FALLBACK_MODEL, default `openai/gpt-oss-20b`). A missing key drops that provider. With no keys, every answer is the template.
- **Truncation is a failure.** Groq answers must finish with `finish_reason == "stop"` and Gemini with `finishReason == "STOP"`; otherwise the next provider is tried.
- **Voice.** `POST /tts` takes the same request body, regenerates (usually from cache) the server's own explanation, and sends that text to ElevenLabs. Clients cannot supply text to speak.

6.3 Share my walk

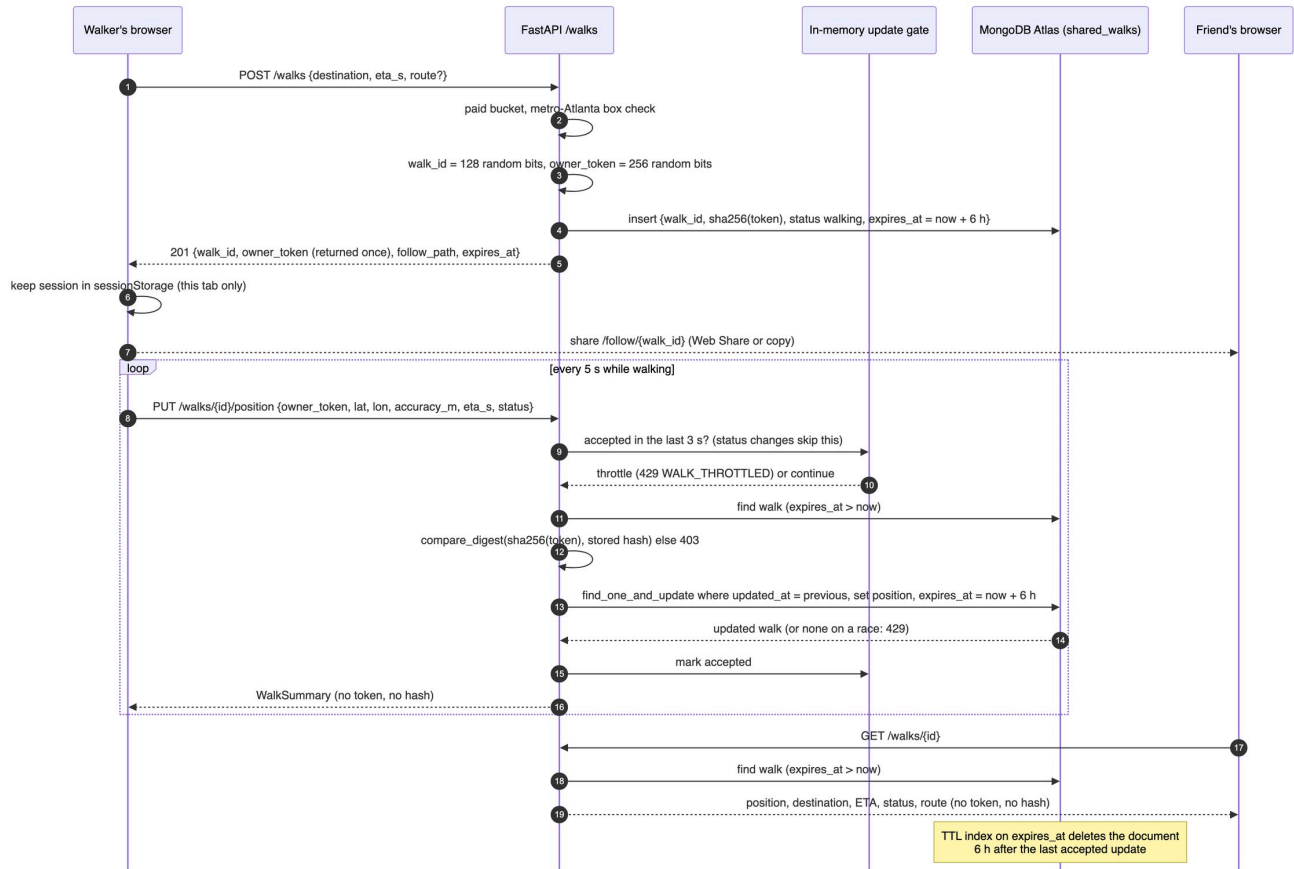


Figure A5. Share-my-walk sequence.

- `PUT /walks/{id}/position` stays on the **general** rate limit on purpose (many walkers share one venue NAT address); the in-memory gate absorbs floods before Atlas is touched. Only authenticated, saved updates mark the gate, so a stranger holding the follow link cannot throttle the walker.
- A walk with status `ended` stays ended; later updates return the summary unchanged.
- In `?demo=1`, sharing is simulated on the device and nothing is sent.

6.4 Offline demo (?demo=1)

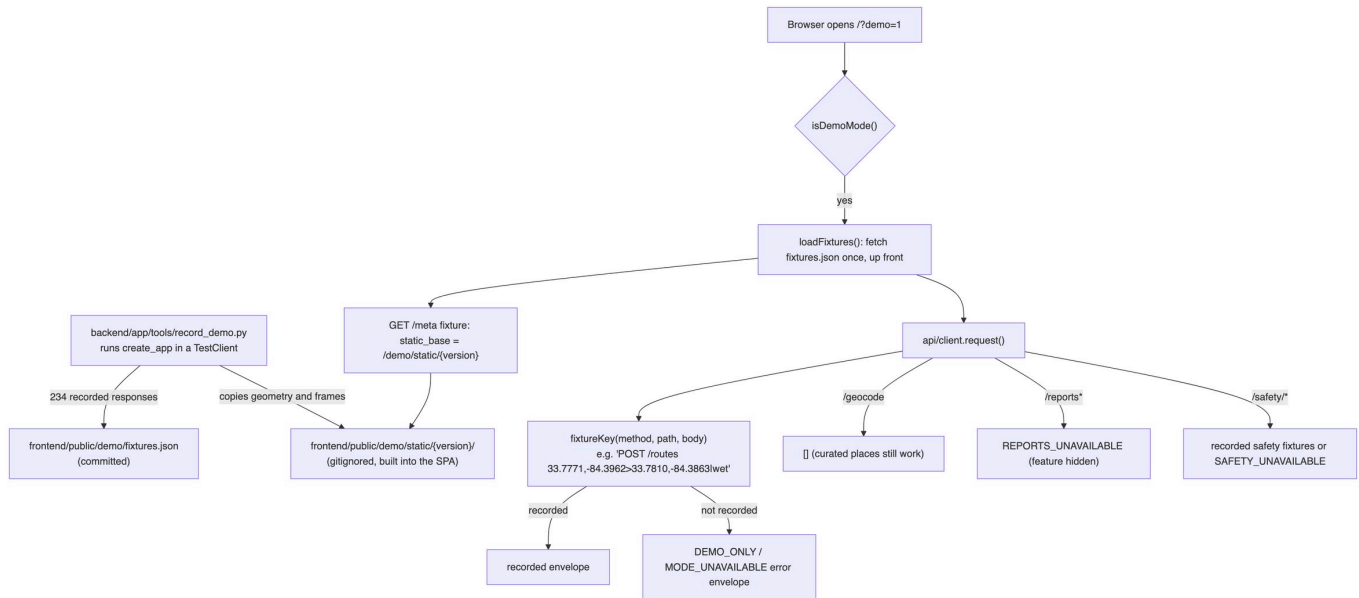


Figure A6. Offline demo transport.

- The scripted scenario is Klaus Building → Midtown MARTA and Tech Square → North Ave MARTA, Friday 22:30, with wet, dry, and live conditions, for walk, bike, e-bike, and scooter.
- Recorded: `/meta`, `/conditions/live`, 24 route responses, 52 explanations, segment details for every top segment, three City Pulse lookups, safety meta, help points, 24 hourly safety hex sets, and MARTA stations (234 keys).
- `frontend/e2e/demo.spec.ts` loads the page, then calls `context.setOffline(true)` and runs the full route, preview-walk, and copy checks with the network off.

7. Multi-mode design (walk vs ride)

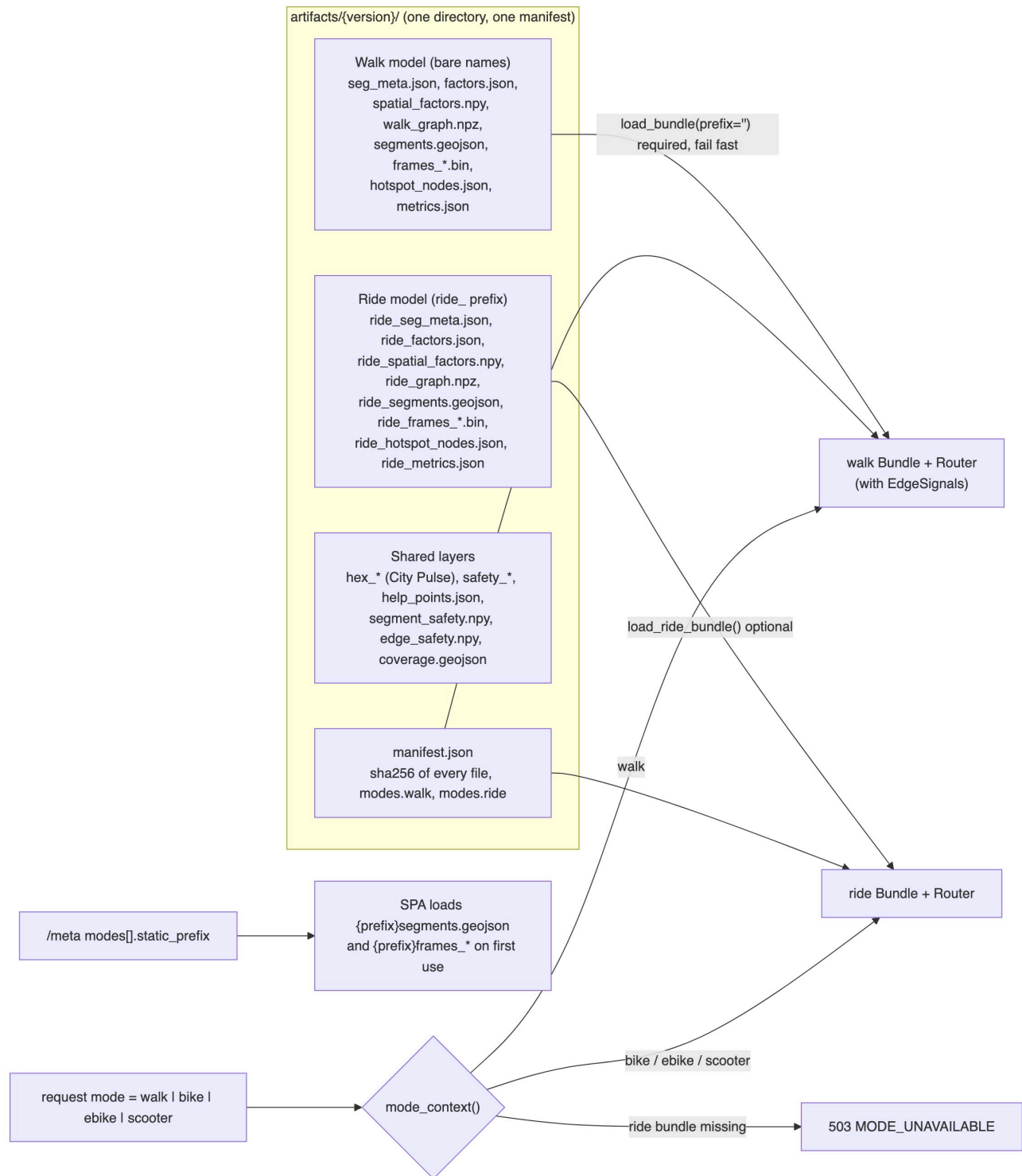


Figure A7. One bundle, two travel networks.

- **Same code path, different prefix.** `load_bundle(root, prefix)` reads `{prefix}seg_meta.json`, `{prefix}factors.json`, `{prefix}spatial_factors.npy`, and `walk_graph.npz` or `ride_graph.npz`. The walk loader verifies every non-ride_ file; the ride loader verifies only ride_ files, so a broken ride file can never block walking.

- **Same risk units.** Both models score the same 49,915 road segments and share segment ids. Only the routing graph (OSMnx walk vs bike network), the training label, the ride-only features, and the frames differ.
- **Mode-specific behaviour.** Ride modes plan at the mode's speed, use the ride long-trip threshold (15 km), skip community reports and the personal-safety summary, ignore the `lit_and_busy` preference, and report `bike_crashes` in segment history. Route cache keys and explanation cache keys add `|mode=...` only for ride modes, so walk keys are unchanged.
- **Frontend.** `/meta` lists four modes with `available`, `speed_kmh`, `network`, and `static_prefix`. `useRideNetwork` downloads `ride_segments.geojson` and `ride_frames_*` only the first time a ride tab is chosen, so walkers never download them.

8. Personal-safety layer

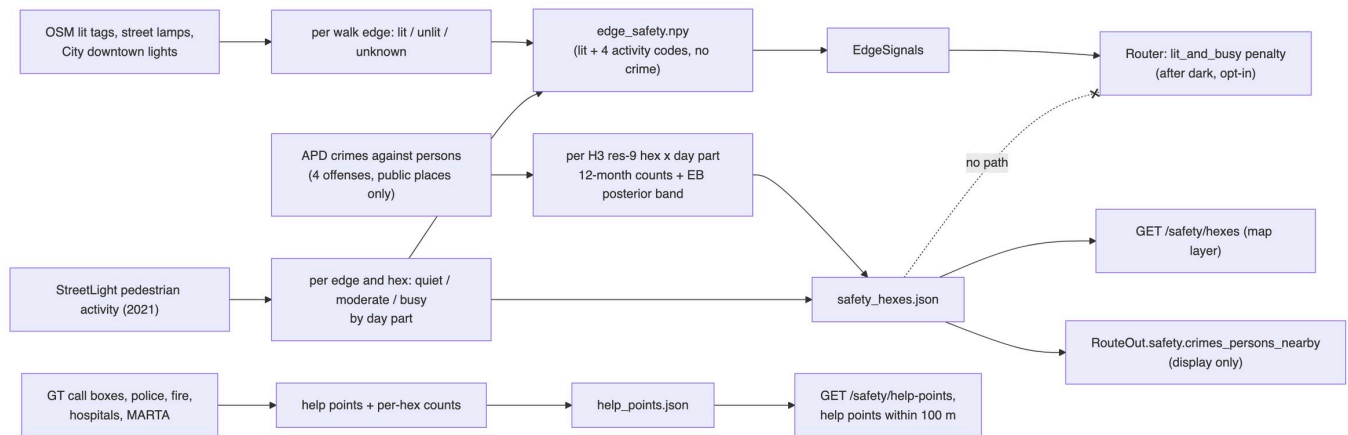


Figure A8. Personal-safety data flow: crime is display-only.

- **What reaches the router.** `EdgeSignals` holds only a lighting code per walk edge and an activity band per day part (`repositories/safety.py`). There is no crime field in it, and a test (`test_crime_counts_never_change_the_route`) builds bundles with crime counts scaled by 0, 1, and 1,000 and asserts that every plan is identical for both preferences.
- **The `lit_and_busy` preference** (walk only, after dark by the same solar-elevation rule as the model): each edge's cost is multiplied by $1 + 1.0 \times [\text{known unlit}] + 0.5 \times [\text{known quiet in the departure's day part}]$. Unknown values are neutral. The ladder is re-run with $\lambda = 0$ plus the usual seven values; candidates must fit the detour budget and add at most 10% traffic exposure over the fastest route; the winner must cut penalty-weighted unlit-or-quiet length by at least 15% against the route the default plan would show, otherwise the default plan is returned unchanged.
- **Route summary.** For walks, `RouteOut.safety` reports `lit_share` and `busy_share` (by length, `null` when under half the route is known), help points within 100 m (polyline densified every 20 m), and `crimes_persons_nearby` (12-month count for the day part over the hexes crossed). The summary is informational.
- **Copy.** Every crime surface carries the fairness note: "Reported incidents, grouped by area and time of day. Reports reflect where police record incidents, not how people should feel about a neighborhood. PathPro never routes around neighborhoods based on crime."

9. Graceful degradation

Dependency	How it fails	What the user sees	Code
------------	--------------	--------------------	------

Walk model bundle	Missing file or SHA-256 mismatch	API does not start (fail fast); systemd restarts it every 2 s; Caddy returns 502 for <code>/api</code> while the SPA still loads	<code>repositories/artifact.s.py</code> , <code>deploy/pathpulse.service</code>
Ride model files	Absent or unreadable	Bike, E-bike, and Scooter tabs show as unavailable; <code>/routes</code> with a ride mode returns 503 <code>MODE_UNAVAILABLE</code> ; walking unaffected	<code>load_ride_bundle</code> , <code>api/deps.mode_context</code>
City Pulse files	Absent	<code>/areas*</code> return 503 <code>CITY_PULSE_UNAVAILABLE</code> ; coverage checks fall back to the bounding box	<code>repositories/hexes.py</code> , <code>services/routing.in_coverage</code>
Safety files	Absent or shape mismatch	<code>/safety/*</code> return 503 <code>SAFETY_UNAVAILABLE</code> ; routes have <code>safety: null</code> ; <code>lit_and_busy</code> behaves like the default	<code>repositories/safety.load_safety</code>
Groq	No key, timeout, HTTP error, truncated or ungrounded text	Next provider; the user still gets a sentence	<code>services/explain.service.py</code>
Gemini	Same	Next provider, then the template	same
All LLMs / daily budget spent	No providers or budget exhausted	Deterministic template text (<code>source: "template"</code>), never cached	<code>template.py</code> , <code>DailyBudget</code>
ElevenLabs	No key or voice, over 420 chars, budget spent, HTTP error	<code>/tts</code> returns 503 <code>TTS_UNAVAILABLE</code> ; the browser speaks with its own voice	<code>services/tts.py</code>
Geoapify	No key, fewer than 3 characters, budget spent, timeout (2.5 s)	<code>/geocode</code> returns <code>[]</code> ; the curated place list (<code>lib/places.ts</code>) still works	<code>services/geocode.py</code>
Open-Meteo	Timeout (4 s) or error	Cached forecast used for up to 1 h; after that "Live weather unavailable — using dry conditions." (<code>source: "assumed"</code>)	<code>services/weather.py</code>
Tiger Data	Unset, unreachable (3 s connect), slow (800 ms statement)	Hourly chart hidden (503 <code>HISTORY_UNAVAILABLE</code>); <code>/healthz</code> reports database and status: <code>degraded</code>	<code>repositories/history.py</code>
MongoDB Atlas (reports)	Unset	Report UI hidden; <code>/reports*</code> 503 <code>REPORTS_UNAVAILABLE</code> ; routes carry <code>reports: []</code>	<code>repositories/reports.py</code>
MongoDB Atlas (reports)	Unreachable or slow	Same, plus a 30 s cooldown after any failure; routes wait at most 0.5 s	same
MongoDB Atlas (walks)	Unset, unreachable, cooling down	503 <code>WALKS_UNAVAILABLE</code> "Live sharing is unavailable right now. Navigation still works."	<code>repositories/walks.py</code> , <code>services/walks.py</code>
Network (client)	Offline or timeout (8 s)	"You're offline. Map still works; routing needs a connection." Frames already fetched keep playing	<code>api/client.ts</code> , <code>frames/frameStore.ts</code>
Browser storage	Blocked or corrupt	Routines and share resume silently disabled; zod-validated reads drop bad data	<code>lib/routines.ts</code> , <code>lib/shareSession.ts</code>

10. Performance

Measure	Value	Source
<code>POST /routes</code> , walk, in-process on a developer Mac (40 trips 0.8–3 km, Friday 22:30, dry, real bundle, no external calls)	See testing_quality.md §6	Measured for this document
Route planning p95 before and after the citywide router refactor	1.95 s → 0.26 s	Commit c8d5e8f message
Default and lit-and-busy plans computed together, 40 random night trips 0.8–2.5 km	p50 154 ms, p95 283 ms	docs/safety_sources.md
Risk Tides frame	24 × 49,915 bytes = 1,197,960 bytes per (day group, condition); City Pulse 24 × 3,537 = 84,888 bytes	Bundle files
Frame swap after first fetch	No network: <code>subarray</code> view per hour	frames/frameStore.ts

PRD targets for reference: `POST /routes` p95 < 1.5 s (NFR-03); frame swap < 100 ms (NFR-02).

Data and models

This document covers the offline pipeline in `data/src/pathpulse_data` (uv package `pathpulse-data`): which public data it pulls, how crashes are cleaned and attached to streets, how the walk and ride models are fitted and evaluated, and what ends up in the artifact bundle the API serves. Every number here is copied from `docs/metrics.json`, `artifacts/<version>/manifest.json`, `ride_metrics.json`, `hex_meta.json`, or `safety_meta.json` for bundle `pp-20260926-1902-f49c0d2`.

Scope reminder: the model predicts **traffic crashes involving pedestrians** (walk) and **cyclists** (ride). It uses no demographic, income, or crime features (PRD NFR-14). The separate personal-safety layer (section 9) never feeds either model.

1. Pipeline overview

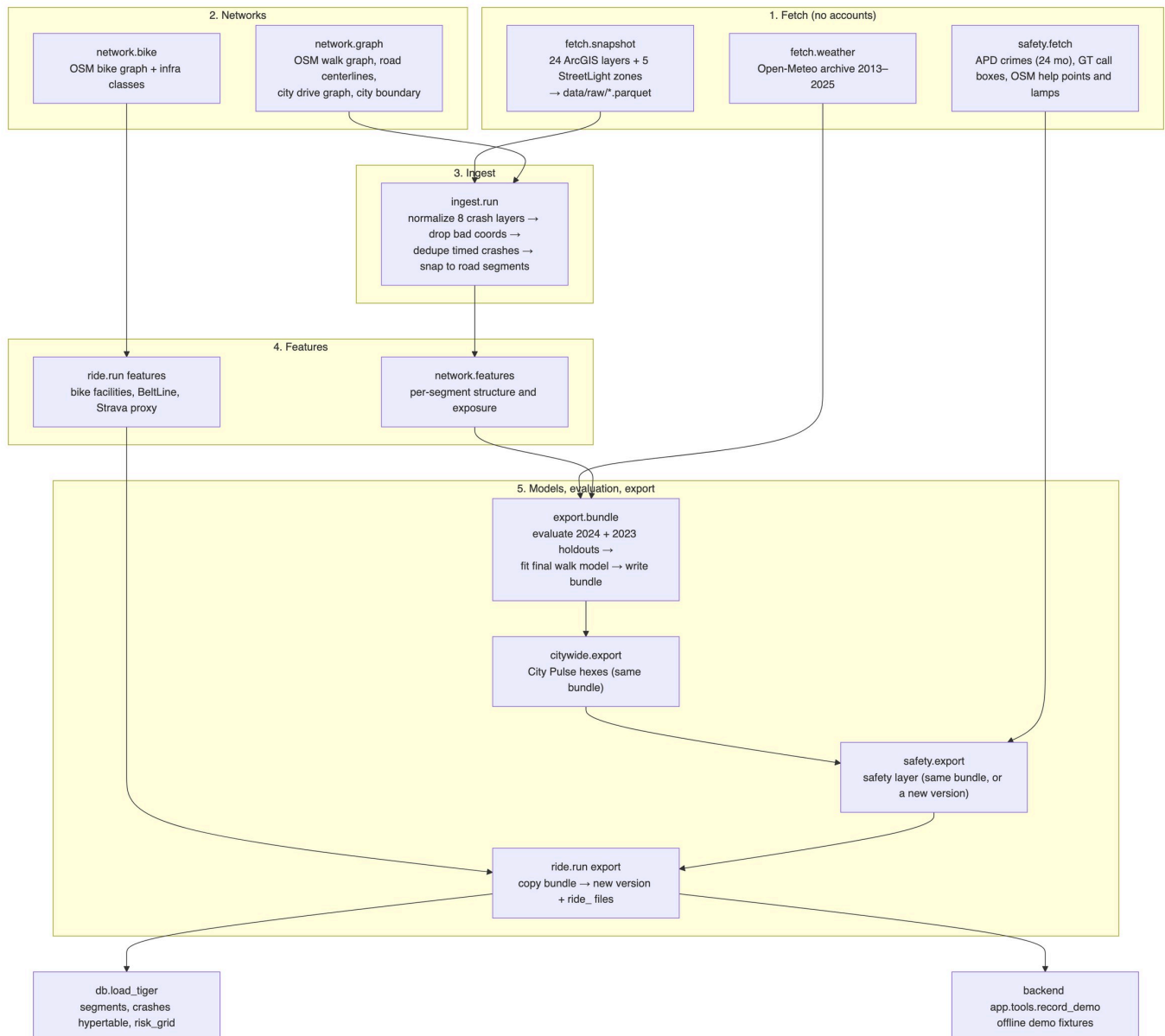


Figure D1. Data pipeline.

All paths are set in `config.py`: `data/raw/` (re-fetchable, gitignored), `data/interim/` (gitignored intermediates), `artifacts/<model_version>/` (bundles; only `artifacts/.gitkeep` is tracked).

2. Data sources

ArcGIS services (from `config.py`):

- `ARC` = `https://services1.arcgis.com/Ug5xGQbHsD8zuZzM/arcgis/rest/services`
- `COA` = `https://services2.arcgis.com/zLeajbicrDRLQcny/arcgis/rest/services`
- `CAP` = `https://services3.arcgis.com/FWC2S7IFSUSD4PZ/arcgis/rest/services`
- `GTMAPS` = `https://services2.arcgis.com/I9cu0JUzvdGAJncI/arcgis/rest/services`

Key	Layer URL (relative to service)	Role
<code>arc_crashes_2020_2024</code>	<code>ARC/Crashes2020_2024/FeatureServer/0</code>	Year-only crashes citywide: spatial model labels (pedestrian flag = pedestrians per crash > 0; cyclist flag = <code>Bicycle_Related</code>) and non-pedestrian crash density
<code>arc_crashes_2019_2023</code>	<code>ARC/Crashes2019to2023/FeatureServer/0</code> (core bbox)	Fetches but not used by <code>ingest.run</code>
<code>coa_all_2022</code>	<code>ARC/COA_2022AllCrashes/FeatureServer/0</code>	Timed crashes, all modes
<code>coa_pedbike_2022</code>	<code>ARC/2022_COA_Pedestrian_and_bicycle_crashes/FeatureServer/0</code>	Timed pedestrian and cyclist crashes
<code>marta_all_2023</code>	<code>ARC/MARTACountyCrashes_2023/FeatureServer/0</code>	Timed crashes with pedestrian and cyclist flags
<code>cap_downtown_2017_2021</code>	<code>CAP/Downtown_Transportation/FeatureServer/9</code>	Timed downtown crashes
<code>coa_midtown_2019_2023</code>	<code>COA/Fiveyear_Crashdata_Midtown_WFL1/FeatureServer/0</code>	Timed Midtown crashes
<code>coa_ka_since_2013</code>	<code>COA/KACrashesSince2013/FeatureServer/0</code>	Timed killed/serious crashes
<code>gt_pedcyc_2021_2025</code>	<code>GTMAPS/Atlanta_Collisions_Involving_Ped_or_Cyclist/FeatureServer/3</code>	Timed pedestrian/cyclist collisions
<code>coa_aadt_2023</code>	<code>COA/SummaryStats_Routes_AADT/FeatureServer/17</code>	Traffic volume
<code>coa_centerline</code>	<code>COA/Centerline_ATLDOT/FeatureServer/0</code>	Speed limit, lanes, ownership
<code>coa_speedlimit</code>	<code>COA/Speedlimit_COA/FeatureServer/0</code>	Speed limit
<code>arc_ped_risk_factors</code>	<code>ARC/Atlanta_Region_Safety_Risk_Factors/FeatureServer/1</code>	Nine structural flags only; income, race, and environmental-justice flags are never read
<code>arc_ped_signals</code>	<code>ARC/PedestrianSignals/FeatureServer/1</code>	Fetches context

coa_sidewalks	COA/Sidewalks_Inventory/FeatureServer/2	Sidewalk presence and condition
coa_marta_bus_stops	COA/MARTA_Bus_Stops_COA/FeatureServer/0	Bus stops and boardings
coa_school_zones	COA/School_Zones_with_Schedules/FeatureServer/0	School zones
coa_downtown_lights	COA/Downtown_Lights_all_WFL1/FeatureServer/0	Street lamps (safety layer)
coa_bike_facilities	COA/Bike_Facilities_Public_View/FeatureServer/0	Ride features (status Existing only)
coa_beltline	COA/Atlanta_BeltLine/FeatureServer/1	BeltLine adjacency (status not Planning)
arc_strava_bike_origins / _destinations	ARC/Strava_Bike_Hexagons_WFL1/FeatureServer/9 and /4	Cycling activity proxy (Strava Metro 2024, H3 res-8)
coa_hin_2025	COA/HIN_Tiers_2025/FeatureServer/0	Baseline only , never a feature
arc_hin_severity	ARC/ARC_High_Injury_Network_Severity/FeatureServer/0	Baseline only
StreetLight zones ZA11-ZA51	COA/Citywide-Pedestrian-Activity--250ftHex--{zone}/FeatureServer/0	Pedestrian activity by day type and day part (2021)

Other sources:

Source	URL	Role
Open-Meteo archive	https://archive-api.open-meteo.com/v1/archive (2013-01-01 to 2025-12-31, hourly precipitation at 33.7756, -84.3963)	Wet/dry exposure hours
OpenStreet Map via OSMnx	Overpass	Walk, bike, and drive networks; signals, crossings, restaurants and nightlife, rail stations; lit tags; help points
APD open data	https://services3.arcgis.com/Et5Qfajgiyosiw4d/arcgis/rest/services/OpenDataWebsite_Crime_view/FeatureServer/0	Safety layer only (section 9)
GT call boxes	https://services5.arcgis.com/7WaXTZEsI88qiQGw/arcgis/rest/services/Call_Box_Location_View_Layer/FeatureServer/0	Safety layer only

The ArcGIS client (`fetch/arcgis.py`) pages with `resultOffset / exceededTransferLimit` (2,000 rows per page), retries four times with exponential back-off, and filters to the city bounding box (`-84.56, 33.64, -84.28, 33.89`) unless a layer overrides it.

3. Ingest: clean, dedupe, snap

`ingest/run.py` produces `crashes_timed.parquet`, `crashes_yearly.parquet`, `crash_segments.parquet`, and `ingest_report.json`.

Normalize (`ingest/clean.py`). Each of the eight crash layers has its own normalizer into one schema: `source`, `source_id`, `collision_id`, `ts`, `year`, `time_precision`, `lat`, `lon`, `is_ped`, `is_bike`, `severity`, `light_report`, `surface_report`, `road`, `cross_road`. Columns that carry personal data (`Name`, `Age`, `Details_411`, `pedestrian`, `driver`, `bicyclist`, and `scooter-rider` ages, `mapUrl`) are dropped or never selected. One source (`coa_pedbike_2022`) stores Atlanta wall-clock time as UTC epoch milliseconds; it is parsed with `wall_clock=True` after its agreement between reported and solar light condition rose from 60% to 94% with that reading.

Validate coordinates. Rows with missing, (0,0), or outside-Georgia coordinates are dropped and counted. For this build no rows were dropped:

Source	Records in	Kept
arc_crashes_2020_2024	251,440	251,440
marta_all_2023	46,613	46,613
coa_all_2022	35,769	35,769
cap_downtown_2017_2021	32,847	32,847
coa_ka_since_2013	6,004	6,004
coa_midtown_2019_2023	5,922	5,922
coa_pedbike_2022	548	548
gt_pedcyc_2021_2025	483	483

Deduplicate timed crashes (`ingest/dedupe.py`). Two timed records are the same crash if they share a GDOT collision id, or lie within 20 m and 30 minutes of each other. Clusters are formed with a disjoint set; a merged record keeps the most severe KABCO letter and ORs the pedestrian and cyclist flags. Result: 127,011 → 122,257 records (4,754 merges). The ARC year-only stream has no collision id, so only exact `OBJECTID` repeats are removed from it.

Snap to road segments (`ingest/snap.py`, in UTM 16N metres). The risk unit is the OSM road centerline segment (interstates excluded), because crashes are geocoded to centerlines and most Midtown sidewalks are separate OSM ways.

1. Within 15 m of an intersection node (degree ≥ 3): split with weight 1/degree across that node's segments.
2. Else the nearest segment within 30 m: weight 1.
3. Else dropped (mostly interstate crashes).

Stream (inside the buffered city boundary)	Crashes	Pedestrian	Snapped	Snapped pedestrian
Year-only (ARC 2020–2024)	174,198	2,337	130,064	2,228 (95.3%)
Timed (all timed layers, deduped)	111,129	2,193	82,305	2,063 (94.1%)

Pedestrian crashes per year in the year-only stream: 2020 388, 2021 463, 2022 435, 2023 490, 2024 561. Data runs through 2026-09-19 (`data_through`).

4. Networks

Network	Builder	Size in this bundle	Use
Road centerlines	<code>network.graph.build_core_road_graph</code> (OSM custom filter: trunk to service roads, no private access, no parking aisles or driveways, no motorways)	49,915 segments	Risk unit for both models
Walk graph	OSMnx <code>network_type="walk"</code> over the city boundary buffered 200 m, largest weakly connected component	84,758 nodes, 120,763 edges	Walk routing
Bike graph	OSMnx <code>network_type="bike"</code> , same polygon, largest component	49,824 nodes, 65,997 edges	Ride routing
City drive graph	OSMnx <code>graph_from_place("Atlanta, Georgia, USA", network_type="drive")</code>	n/a	City Pulse hex features only

Risk inheritance (`network/inherit.py`). Each routing edge takes the `seg_id` of the road it is exposed to, matched by midpoint distance and heading: road edges within 15 m and 30°, footpaths within 25 m and 30°, crossings within 20 m at any angle. Edges with no nearby road (park paths, campus quads, the BeltLine) get `-1` and are scored at the citywide median density. 99,119 of 120,763 walk edges and 53,158 of 65,997 bike edges inherit a road.

Edge geometry orientation. OSMnx stores many edge geometries in the opposite direction; the API orients each edge's vertices from the node being left (`route_metrics.oriented_coords`), a fix from commit `f99e683` that made drawn routes match their true length.

5. Features (per road segment)

`network/features.py` conflates agency lines onto segments by midpoint distance (≤ 20 m) and heading ($\leq 30^\circ$) so a cross street never donates its attributes.

Factor shown to users	Model columns
Traffic volume	<code>log_aadt</code> , <code>aadt_missing</code> , <code>arc_aadt_risk</code>
Speed limit	<code>speed</code> (City speed-limit layer, then centerline, then OSM <code>maxspeed</code>), <code>arc_psl_risk</code>
Number of lanes	<code>lanes</code> (OSM, then centerline), <code>arc_lanes_risk</code>
Road type	<code>group_arterial/collector/local</code> , <code>is_service</code> , <code>oneway</code> , <code>state_owned</code> , <code>arc_gdot_risk</code> , <code>arc_min_art_risk</code> , <code>arc_urban_risk</code> , <code>arc_high_dev_risk</code> , <code>arc_structural_count</code> (+ the temporal model's road-group level)
Pedestrian activity	<code>log_ped_volume</code> (StreetLight all-day volume within 60 m), <code>ped_volume_missing</code>
Restaurants and nightlife nearby	<code>log_food_n</code> , <code>log_nightlife_n</code> (OSM amenities within 150 m)
Transit stops	<code>bus_stops_n</code> (40 m), <code>log_bus_boardings</code> , <code>arc_bus_risk</code> , <code>arc_high_bus_risk</code> , <code>log_rail_dist</code>
Intersection complexity	<code>node_degree_max</code> , <code>signals_n</code> (15 m), <code>crossings_n</code> (10 m)
Sidewalk condition	<code>sidewalk_n</code> , <code>sidewalk_cond</code> (25 m)
School zone	<code>school_zone</code> (60 m)

Mapped street lighting	<code>osm_lit</code>
Vehicle crashes on this street	<code>log_nonped_density</code> , <code>log_nbr_nonped_density</code> (training years only)
Pedestrian crashes on nearby streets	<code>log_nbr_ped_density</code> (other segments within 200 m, training years only)
Block length	<code>log_len</code> (length clipped at 30 m)
Pedestrian crash history here	Empirical Bayes adjustment (section 6.2)

Missing numeric values are filled with the road-group median. Crash-derived features are recomputed for each training window from that window's years only, so a holdout year never leaks into its own features.

6. Walk model

The expected annual pedestrian crash density of a segment in a time cell is the product of a **spatial** model (where) and a **temporal** multiplier (when), with an exact additive split in log space.

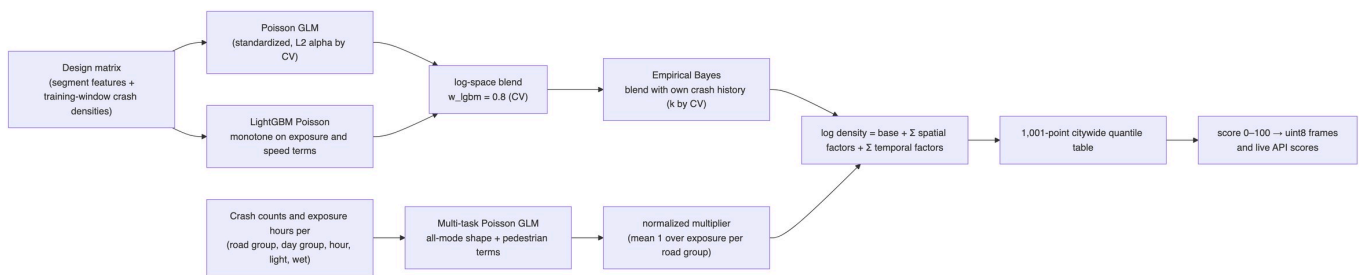


Figure D2. Model composition.

6.1 Spatial model (safety performance function)

- **Target:** annual pedestrian crash rate per segment (`window_counts` / `n_years`), sample weight `n_years`, from the snapped ARC year-only stream.
- **GLM member:** scikit-learn `PoissonRegressor` on standardized columns; `L2_alpha` chosen from {0.003, 0.01, 0.03, 0.1, 0.3, 1.0} by grouped 5-fold CV deviance.
- **LightGBM member:** Poisson objective, learning rate 0.03, 7 leaves, 60 minimum rows per leaf, 0.8 feature and bagging fractions, `L2` 5.0, seed 13. Monotone-increasing constraints on `log_aadt`, `lanes`, `speed`, `log_nonped_density`, `log_ped_volume`, `log_len`, `log_nbr_ped_density`, `log_nbr_nonped_density` (and `log_bike_activity` for ride). Boosting rounds chosen by `lgb.cv` with early stopping (100) up to 1,500.
- **Spatial-block CV:** every fold split is `GroupKFold(5)` over H3 resolution-7 blocks (~5 km²), coarse enough that the split pieces of one intersection crash stay in one fold.
- **Ensemble:** $\log \mu = w \cdot \log \mu_{\text{lgbm}} + (1 - w) \cdot \log \mu_{\text{glm}}$, with `w` on a 0.1 grid chosen on out-of-fold predictions.
- **Selected on the 2020–2023 fit:** `glm_alpha` 0.003, 543 rounds, `w_lgbm` 0.8, CV deviance 0.0358 (validation fit 2020–2022: 521 rounds, same alpha and weight).

6.2 Empirical Bayes

Highway Safety Manual form (`model/eb.py`): $EB = w \cdot \mu + (1 - w) \cdot y$ with $w = 1 / (1 + k \cdot \mu)$, where μ and y are totals over the same window. k is chosen from $\{0, 0.1, 0.25, 0.5, 1, 2, 4, 8, 16, 32\}$ by predicting the last training year from the earlier ones (method-of-moments k is biased toward 0 here because intersection crashes are split into fractional counts). Selected: $k = 0.25$ (test fit), 0.5 (validation fit).

6.3 Temporal model

`model/temporal.py` fits one `PoissonRegressor` (alpha 1e-3) on stacked cells: (road group $\times$ day group $\times$ hour $\times$ light $\times$ wet) $\times$ {all-mode, pedestrian}, with rates per 10,000 exposure hours weighted by hours, plus year indicators (and pedestrian $\times$ year) that absorb changes in which layers exist in which year.

- **Shared terms:** road group, day group (weekday / Friday / Saturday / Sunday), 24 hours, light (day / twilight / dark), wet.
- **Pedestrian terms:** light, five hour bins (0–5, 6–9, 10–14, 15–18, 19–23), day group, road group, wet.
- **Same definitions for crashes and exposure** (`timeseries/exposure.py`): light from solar elevation at 33.7756, –84.3963 at mid-hour (day $> -0.833^\circ$, twilight $> -6^\circ$, else dark); wet when precipitation ≥ 0.1 mm in the current or previous hour; DST-exact hour starts.
- **Output:** a multiplier normalized so its exposure-weighted mean is 1 within each road group, so the spatial model keeps its annual scale.
- **Windows:** evaluated by training on 2017–2021 and testing on 2022–2023; the shipped model is fit on 2017–2023 (2024–25 timed data comes from only two narrow layers).

6.4 Exact factor attribution

`export/factors.py` groups per-feature log contributions into the 15 spatial and 4 temporal factors users see:

- LightGBM contributions come from `predict(pred_contrib=True)` (TreeSHAP); GLM contributions are $z \cdot \text{coef}$; both are blended with the same w , so the sum is exact.
- The Empirical Bayes adjustment $\log(EB / \text{SPF})$ is added to **Pedestrian crash history here**; $-\log(\text{length} / 100 \text{ m})$ to **Block length** (scores are per 100 m); the temporal model's road-group level to **Road type**.
- Every factor is centered on its mean, so `base` means "a typical street at a typical hour".
- `export/assemble._check_identity` asserts that the decomposed log density equals $\log(EB \times \text{multiplier} / \text{length})$ within 1e-6 for Friday 22:00 wet, or the export fails.

At request time, `domain/scoring.attribute` turns log contributions into integer score points by adding factors from largest to smallest `|effect|` and assigning each the change in percentile score it causes; the top five are shown and the rest become `remainder_points`, so **baseline + bars + remainder = score** exactly.

6.5 Score scale and Risk Tides frames

- **Quantiles:** 1,001 quantiles of log density over every segment $\times$ 24 hours $\times$ 8 frames (4 day groups $\times$ dry/wet). The score is the piecewise-linear percentile on that table, so "High" (75+) always means the top quarter of the city's street-hours for this model version.
- **Reference dates:** each day group uses its next date on or after the build day (Friday 2026-09-25, Saturday 09-26, Sunday 09-27, weekday 09-28); light for hour h is computed at $h:30$ on that date and recorded in `manifest.frame_light`. The API scores live requests with the exact light at the requested time.
- **Format:** `frames_{day}_{cond}.bin` is `uint8`, hour-major `[24] [n_segments]`, scores rounded and clipped to 0–100. The browser indexes it by segment id.
- **Confidence:** high when a segment has ≥ 15 weighted crashes (all modes, 2020–2024), `medium` ≥ 3 , else `limited`.

- **Hotspots:** `hotspot_nodes.json` lists 26,486 road intersections (degree ≥ 3) with their segment ids; the SPA glows the top 5% by maximum incident score, never below 75.

7. City Pulse (citywide hexes)

`citywide/` repeats the approach on 3,537 H3 resolution-9 cells ($\sim 0.1 \text{ km}^2$) whose centers fall inside the City of Atlanta boundary.

- **Features per hex:** km of arterial, collector, and local road (city drive graph), intersections, signals, mean `log_aadt`, mean speed limit, bus stops, `log_boardings`, mean `log_ped_volume` (with missing flags), `log_nonped`, and 1-ring neighbour pedestrian and non-pedestrian crash sums (training years only).
- **Labels:** ARC year-only crashes located in each hex (not snapped).
- **Fit:** GLM (alpha 0.01) + monotone LightGBM with rounds chosen by CV grouped on H3 resolution-6 parents; the blend weight is chosen in-sample on the training window; EB k by the last-year check.
- **Time and conditions:** a road-length-weighted mixture of the pedestrian temporal multipliers, shown as a single factor "Time of day and conditions".
- **Outputs:** `hex_meta.json` (cells, centroids, road-group shares, crash counts, confidence ≥ 30 / ≥ 5 crashes, base, quantiles, factor labels, multipliers, metrics), `hex_factors.npy` ($3,537 \times 7$), `hex_frames_{day}_{cond}.bin` ($24 \times 3,537$ bytes), and `hex_cells.json`.

8. Ride model (bike, e-bike, scooter)

`ride/run.py` reuses the walk pipeline on the same 49,915 segments with a different target, extra features, and its own routing graph.

- **Target flag:** `is_bike` (ARC `Bicycle_Related`; 2020–2024: 29, 95, 123, 132, 175 crashes by year; 551 of 554 in coverage snapped). Scooter riders appear only in the killed/serious layer and are not counted as cyclists.
- **Extra features** (`ride/features.py`): strongest bike facility within 20 m and 30° from OSM cycleway tags and the City's `Bike_Facilities_Public_View` (protected / painted / shared, one-hot); BeltLine adjacency within 15 m (OSM names or the ABI layer); `log_bike_activity` from Strava Metro 2024 ride and e-bike trip origins plus destinations in the segment's H3 res-8 hex. Segment counts by facility class: none 46,012, shared 611, painted 1,885, protected 1,407.
- **Exposure is a proxy.** The City publishes no StreetLight bicycle layer. Without the Strava feature the 2024 capture is 69.5% [63.9, 75.8] (model card), so Strava does not drive the result.
- **Training label chosen on validation.** Two candidates were fit and scored on **cyclist crashes only**: cyclist-only labels, and pooled pedestrian + cyclist labels. The pooled label won on the 2023 validation year and is shipped.

Candidate (scored on cyclist crashes)	2023 validation, top 10%	2024 test, top 10% [95% CI]	2024 gain over reusing the walk model [95% CI]
Cyclist-only	59.7%	57.4% [48.1, 65.6]	−16.4 to −0.6 points
Pedestrian + cyclist (shipped)	69.0%	69.9% [64.0, 76.1]	+1.3 to +6.2 points

- **Temporal choice.** Three temporal structures were compared on held-out 2022–2023 cyclist crashes (shape deviance): cyclist-specific 0.516, all-mode shape 0.562, walk structure reused 0.580. The cyclist-specific model (trained on 48 cyclist crashes from 2017–2021 plus the all-mode shape) was chosen and refit on 2017–2023. It matches a smoothed hour $\times$ day baseline (−0.06%), so light and rain add essentially nothing for cyclists.
- **Bundle safety.** `ride/bundle.new Ride version` copies the source bundle byte for byte into a new version directory, writes only `ride_*` files, and `verify_unchanged` fails the export if any walk file's bytes changed.

- **Speeds.** Bike 15 km/h, e-bike 22 km/h, scooter 18 km/h (`backend/app/domain/modes.py`). All three share the ride model.

9. Personal-safety layer

Built by `safety/export.py` from `safety/fetch.py` pulls. Full method and coverage are in `docs/safety_sources.md`; the essentials:

Signal	Method	Coverage in this bundle
Reported crimes against persons	APD NIBRS homicide, robbery, aggravated assault, simple assault; excludes residences, apartments, jails, and shelters; fetches only <code>OccurredFromDate</code> , <code>NIBRS_Offense</code> , <code>LocationType</code> + point; converts true UTC to Atlanta time; counts per H3 res-9 hex × day part	15,096 raw rows; 6,174 kept for 24 months (banding), 3,268 for 12 months (shipped counts), 3,167 inside city hexes
Crime band	Exposure-normalized Empirical Bayes: E = expected count if every pedestrian-hour had the city rate (activity × hours in day part; missing activity → city median); $\text{Gamma}(\alpha, \alpha/m)$ prior by method of moments (Clayton & Kaldor 1987); posterior $\text{Gamma}(y + \alpha, E + \alpha/m)$. "higher" if $P(\theta > m) \geq 0.9$ and $y > 0$; "lower" if $P(\theta < m) \geq 0.9$; else "typical"	Night 701 / 2,614 / 222 (lower / typical / higher); morning 439 / 2,960 / 138; afternoon 642 / 2,662 / 233; evening 465 / 2,890 / 182
Lighting	Edge's own OSM <code>lit</code> tag, else its road segment's tag, else a mapped lamp within 25 m counts as lit; only <code>lit=no</code> or <code>disused</code> is unlit; hex <code>lit_share</code> is null unless $\geq 50\%$ of walkable length is known	Known for 4.4% of walkable length; 56 of 3,537 hexes have a <code>lit_share</code> ; 2,523 lamps
Foot traffic	StreetLight daily volume per period → mean hourly rate per day part; bands are pooled terciles: quiet < 2.56/h, moderate 2.56–9.06/h, busy ≥ 9.06 /h; segments blend weekday and weekend 5:2	3,498 hexes; 43,621 segments
Help points	GT call boxes (active, outdoor), OSM police, fire, hospitals, MARTA rail; same-kind points within 30 m merged	189 points: 100 blue-light, 35 fire, 24 MARTA, 21 police, 9 hospital

Day parts (Atlanta time): night 22–05, morning 06–11, afternoon 12–17, evening 18–21.

10. Evaluation

10.1 Protocol

- **Temporal holdout.** Train on 2020–2023, test on 2024 pedestrian crashes. Second check: train 2020–2022, test 2023. Features that use crash history are rebuilt from the training window only.
- **Headline metric:** share of held-out crashes on the top X% of **street length** when segments are ranked by predicted density per 100 m. The segment straddling the budget counts pro-rata. Ranking by length stops a model from winning by picking long segments.
- **Uncertainty:** 95% percentile intervals from 400 bootstrap resamples of 614 H3 resolution-8 spatial blocks (whole blocks are resampled, because nearby streets are not independent). The gain over the past-crash baseline has its own paired block-bootstrap interval.
- **Baselines** (`model/benchmark.py`): past pedestrian crash density in the training window; the City's 2025 High Injury Network tier; ARC structural risk flags (demographic flags removed); random. Tied baseline scores get a seeded $1e-9$ jitter so ties are not broken by row order. The HIN is also compared at its own share of street length (9.8%).
- **Other metrics:** ROC-AUC and PR-AUC for "segment has ≥ 1 crash in the holdout", mean Poisson deviance, and calibration by decile.

10.2 Walk model, 2024 holdout (543 pedestrian crashes)

Method	Top 10% length	Top 5% length	At HIN's 9.8%	ROC-AUC	PR-AUC
PathPro (EB ensemble)	74.3% [70.5, 78.3]	61.1%	73.9%	0.889	0.277
Model only (SPF, no EB)	74.3%	61.2%	73.8%	0.889	0.276
Past pedestrian crash density	49.8%	44.4%	49.8%	0.716	0.160
City High Injury Network 2025	53.8%	39.3%	53.7%	0.694	0.088
ARC structural flags (demographic flags removed)	51.2%	26.2%	50.1%	0.792	0.085
Random	11.9%	6.1%	11.6%	0.494	0.027

- Gain over past-crash ranking: +24.5 points, 95% CI [+20.4, +28.8].
- Poisson deviance: EB 0.0515, SPF 0.0516, count-only 0.1523.
- Predicted 411.0 vs observed 543 crashes: counts are under-predicted, so the claim is **ranking**, not calibrated counts.

10.3 Walk model, 2023 check (train 2020–2022; 475 pedestrian crashes)

Method	Top 10% length	Top 5%	At HIN share	ROC-AUC	PR-AUC
PathPro (EB ensemble)	68.1% [63.9, 72.4]	58.0%	67.5%	0.869	0.248
Model only (SPF)	68.0%	57.3%	67.8%	0.869	0.248
Past pedestrian crash density	45.5%	41.4%	45.2%	0.682	0.127
City High Injury Network 2025	54.2%	38.2%	53.9%	0.703	0.082
ARC structural flags	50.8%	27.1%	50.3%	0.793	0.079
Random	12.4%	6.3%	12.3%	0.508	0.026

Gain over past-crash ranking: +22.6 points, 95% CI [+18.1, +26.9]. Deviance: EB 0.0486, SPF 0.0487, count-only 0.1490. Predicted 393.4 vs observed 475.

10.4 Calibration by decile of predicted value (observed / predicted)

Decile	2024: predicted	2024: observed	Ratio	2023: predicted	2023: observed	Ratio
0 (lowest)	1.1	0.7	0.60	1.1	1.2	1.09
1	1.9	2.3	1.23	1.9	2.8	1.47
2	3.2	4.3	1.35	3.1	6.5	2.12
3	4.4	1.2	0.28	4.6	4.0	0.87
4	5.8	6.2	1.06	6.0	5.9	0.98
5	8.3	16.3	1.97	8.3	10.8	1.29
6	13.4	24.8	1.85	12.9	21.2	1.64
7	25.4	42.7	1.68	23.4	50.4	2.15
8	54.4	67.1	1.23	51.4	55.6	1.08
9 (highest)	293.0	377.3	1.29	280.7	316.7	1.13

10.5 Temporal model (held-out 2022–2023, 889 pedestrian crashes)

Shape deviance (each prediction rescaled to the test total, so only the time profile is compared):

Model	Deviance	Reduction
Flat (rate constant within road group)	1.0334	baseline
Smoothed hour × day GLM (no light, no rain)	0.7902	23.5% vs flat
PathPro temporal GLM	0.7801	24.5% vs flat; 1.3% vs hour × day

Pedestrian multipliers from the evaluation fit (2017–2021): dark ×1.62, twilight ×1.44, wet ×1.12, pedestrian-specific darkness term ×1.11. From the shipped fit (2017–2023, `temporal_effects`): dark ×1.63, twilight ×1.35, wet ×1.04, pedestrian × dark ×1.28.

10.6 Ride model (scored on cyclist crashes)

Method	2024 top 10% (174 crashes)	2024 ROC-AUC	2023 top 10% (132 crashes)	2023 ROC-AUC
Ride model (EB ensemble, pooled label)	69.9% [64.0, 76.1]	0.866	69.0% [62.3, 76.6]	0.867
Model only (SPF)	69.7%	0.866	69.4%	0.867
Walk model reused	66.4%	0.855	62.5%	0.851
City High Injury Network 2025	43.6%	0.642	48.9%	0.691
ARC structural flags	42.4%	0.769	43.9%	0.789
Past cyclist crash density	30.4%	0.611	22.1%	0.569
Random	12.0%	0.501	11.8%	0.480

- 2024 gains: over past cyclist crashes +30.9 to +46.3 points; over the reused walk model +1.3 to +6.2 points (95% CIs).
- Pre-registered targets (`ride/evaluate.targets`): CI lower bound above the count-only point estimate, and above twice the random capture. Both met.
- Temporal: 25.1% lower deviance than flat on 252 held-out cyclist crashes (0.516 vs 0.689). Shipped multipliers: dark ×1.00, twilight ×1.08, wet ×0.86 (exposure is clock hours, not riders).

10.7 City Pulse (top 10% of hexes, equal weight per hex; no bootstrap interval computed)

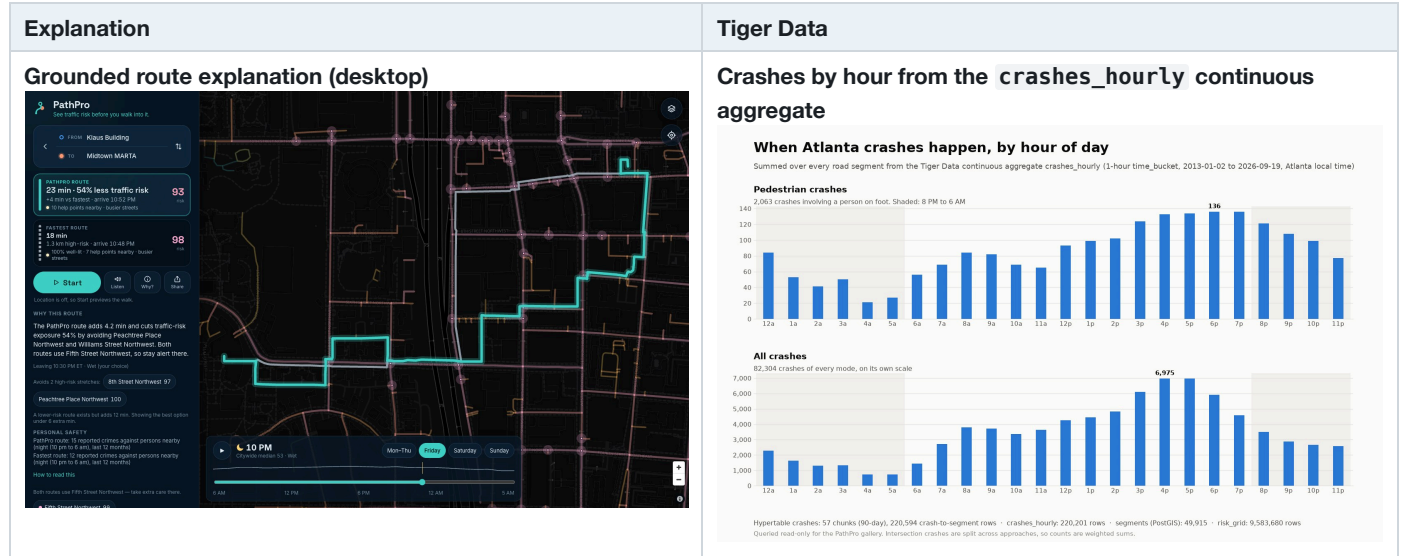
Method	2024 (548 crashes)	ROC-AUC	2023 (468 crashes)	ROC-AUC
City Pulse (EB ensemble)	74.5%	0.916	69.2%	0.908
Past crash count only	66.1%	0.829	58.8%	0.765
Random	13.1%	0.470	13.0%	0.465

10.8 Caveats carried from the model card

- Ranking a whole city (with many quiet residential streets) is easier than ranking a dense core; the earlier core-only model reached 45.8% vs 41.4% for past crashes.
- 2024 was looked at during development; the 2023 check is the cleaner second opinion.
- The HIN targets killed and serious crashes of all modes, so the comparison is conservative against PathPro.

- Street features (OSM signals and crossings, 2023 AADT, current speed limits, current bike facilities) are snapshots taken during or after the test period.
- Exposure bias: busy streets score higher partly because more people walk there. StreetLight activity is from 2021.

Scores are split into factors as in section 6.4 and put into words as in [architecture §6.2](#); the hourly crash chart comes from Tiger Data (section 12):



11. Bundle layout

A bundle is one directory, `artifacts/<model_version>/`, where `model_version` = `pp-<UTC yyyyymmdd-HHMM>-<git short sha>`. `artifacts/current` is a relative symlink to the served version. `manifest.json` records the SHA-256 of every other file; the API refuses to start on a mismatch in any walk file.

File	Format	Purpose	Read by
<code>manifest.json</code>	JSON	Version, <code>data_through</code> , <code>n_segments</code> , day groups, conditions, reference dates, <code>frame_light</code> , coverage bbox, walk/ride graph stats, <code>n_hexes</code> , lineage (derived from, <code>safety_added_at</code> , <code>ride_added_at</code>), modes, and files → sha256	API
<code>metrics.json</code>	JSON	Walk evaluation (<code>spatial_test</code> , <code>spatial_validation</code> , <code>temporal_test</code>), headline, <code>temporal_effects</code> , ingest report	API (<code>/meta.headline</code>)
<code>seg_meta.json</code>	JSON columns	Per segment: <code>name</code> , <code>road_group</code> , <code>length_m</code> , <code>eb</code> , <code>spf</code> , <code>crashes</code> , <code>ped_crashes</code> , <code>dark_share</code> , <code>wet_share</code> , <code>confidence</code>	API
<code>factors.json</code>	JSON	<code>base</code> , spatial and temporal factor keys and labels, 576 <code>temporal_rows</code> (4 day groups × 24 h × 3 light × 2 wet → 4 values), 1,001 quantiles	API
<code>spatial_factors.npy</code>	float32 [49,915 × 15]	Centered spatial log contributions per segment	API
<code>walk_graph.npz</code>	NumPy archive	<code>node_lon/lat</code> [84,758], <code>edge_u/v/len/seg/kind</code> [120,763], <code>coords</code> [429,275 × 2], <code>coord_offsets</code>	API router

<code>frames_{weekday, friday, saturday, sunday}_{dry, wet}.bin</code>	uint8 [24 × 49,915]	Risk Tides scores	Browser
<code>segments.geojson</code>	GeoJSON	Segment lines (5-decimal coordinates) with <code>n</code> name, <code>g</code> road-group letter, <code>c</code> confidence letter; feature id = seg id	Browser
<code>hotspot_nodes.json</code>	JSON	[<code>lon</code> , <code>lat</code> , [<code>seg_ids</code>]] for 26,486 intersections	Browser
<code>coverage.geojson</code>	GeoJSON	Simplified City of Atlanta boundary (~50 m tolerance)	Browser
<code>hex_meta.json</code> , <code>hex_factors.npy</code> , <code>hex_cells.json</code> , <code>hex_frames*.bin</code>	JSON, float32 [3,537 × 7], JSON, uint8 [24 × 3,537]	City Pulse model, factors, cell list, frames	API, browser
<code>safety_meta.json</code>	JSON	Sources, crime categories, excluded locations, day parts, columns, method, stats	API
<code>safety_hexes.json</code>	JSON columns	Per hex: 12-month crime counts and bands per day part, <code>lit_share</code> , activity band per day part, help-point count	API
<code>help_points.json</code>	JSON	189 points: <code>kind</code> , <code>name</code> , <code>lat</code> , <code>lon</code>	API
<code>segment_safety.npy</code>	int16 [49,915 × 6]	<code>lit</code> , activity per day part ×4, <code>help_dist_m</code>	API (loaded; not used by an endpoint yet)
<code>edge_safety.npy</code>	int8 [120,763 × 5]	<code>lit</code> , activity per day part ×4 (no crime)	API router
<code>ride_seg_meta.json</code>	JSON columns	As <code>seg_meta.json</code> plus <code>bike_crashes</code> , <code>bike_infra</code>	API
<code>ride_factors.json</code> , <code>ride_spatial_factors.npy</code>	JSON, float32 [49,915 × 17]	Ride factors (adds bike facility and cycling activity)	API
<code>ride_graph.npz</code>	NumPy archive	Bike routing graph (49,824 nodes, 65,997 edges), same layout as the walk graph	API router
<code>ride_frames*.bin</code> , <code>ride_segments.geojson</code> (adds <code>b</code> facility code), <code>ride_hotspot_nodes.json</code>	as walk	Ride Risk Tides	Browser
<code>ride_metrics.json</code>	JSON	Ride evaluation, label and temporal choices, sources, facility counts	API (<code>/meta.ride_model</code>)

Bundle lineage on disk:

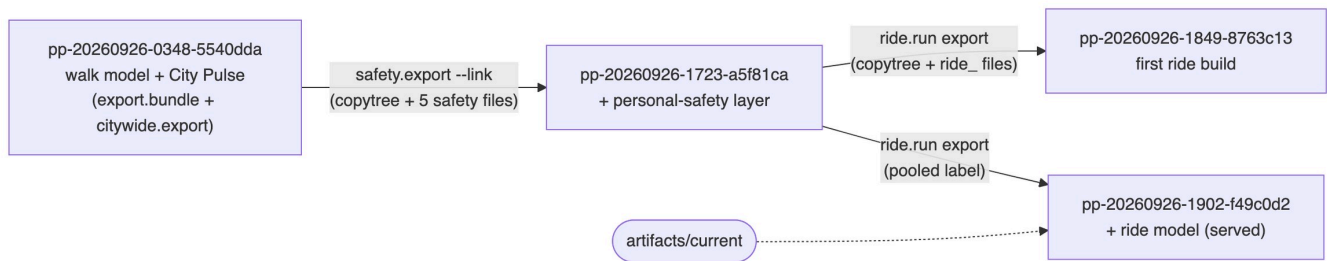


Figure D3. Bundle lineage.

Each extension copies its parent and adds files, so older bundle directories stay valid and can be re-pointed to for rollback.

12. Tiger Data (system of record)

`data/sql/schema.sql` (idempotent) and `db/load_tiger.py`:

Object	Definition	Loaded rows (per sponsor checklist)
segments	seg_id PK, name, road group, length, geometry(LineString, 4326) with a GiST index	49,915
crashes	Hypertable on ts, 90-day chunks; one row per crash-segment assignment (weight, pedestrian flag, severity, light, surface, point geometry); index (seg_id, ts DESC)	220,594 rows, 57 chunks
crashes_hourly	Continuous aggregate: time_bucket('1 hour'), seg_id, sum(weight), pedestrian and dark sums; refreshed after load	n/a
citywide_hour_profile	View: hour-of-day profile in Atlanta time	n/a
risk_grid	Scores per (model_version, day group, condition, hour, segment)	9,583,680

The API reads only `crashes_hourly` (GET `/segments/{id}/hourly`). The load is manual (`LOAD_TIGER=1 deploy/go.sh ...` or the module directly).

13. Rebuilding

Prerequisites: Python 3.12 with uv, Node 22; network access to the public layers.

```

uv sync --all-packages

# Walk model + City Pulse + (if safety raw pulls exist) safety layer, in one bundle
uv run --package pathpulse-data python -m pathpulse_data.fetch.snapshot      # data/raw/*.parquet
uv run --package pathpulse-data python -m pathpulse_data.fetch.weather
uv run --package pathpulse-data python -m pathpulse_data.network.graph        # OSM graphs, boundary
uv run --package pathpulse-data python -m pathpulse_data.ingest.run           # clean, dedupe, snap
uv run --package pathpulse-data python -m pathpulse_data.network.features
uv run --package pathpulse-data python -m pathpulse_data.safety.fetch         # optional layer inputs
uv run --package pathpulse-data python -m pathpulse_data.export.bundle        # evaluate, fit, export, link
current

# Add the safety layer to an existing bundle as a new version instead
uv run --package pathpulse-data python -m pathpulse_data.safety.export --link

# Ride model as a new version derived from current
uv run --package pathpulse-data python -m pathpulse_data.network.bike
uv run --package pathpulse-data python -m pathpulse_data.ride.run features
uv run --package pathpulse-data python -m pathpulse_data.ride.run export
uv run --package pathpulse-data python -m pathpulse_data.ride.run link <new-version>

# Optional: system of record and offline demo
DATABASE_URL=... uv run --package pathpulse-data python -m pathpulse_data.db.load_tiger
cd backend && uv run python -m app.tools.record_demo      # fixtures.json + demo static files

```

Notes:

- `ingest.run` must run after `network.graph` (it snaps to `road_segments.parquet`); `network.features` after both.
- OSMnx graphs are cached as `data/interim/*.graphml`; delete them to force a fresh OSM pull.
- A build produces a new version directory; nothing is overwritten except the `current` symlink.
- The model card figures (CIs, deviances) are regenerated by `export.bundle` (walk), `citywide.export` (City Pulse), and `ride.run export` (ride); `docs/metrics.json` is a copy of the walk `metrics.json` plus a ride summary.

PathPro API reference

Derived from `backend/app/api/*.py`, `backend/app/api/schemas.py`, `backend/app/api/walk_schemas.py`, and the services they call. Sample responses are trimmed copies of real responses (recorded demo fixtures in `frontend/public/demo/fixtures.json`, or read-only calls to the live site).

Base URLs

Environment	Base URL	Notes
Production	<code>https://pathpro.tech/api</code>	Caddy strips <code>/api</code> and proxies to uvicorn on <code>127.0.0.1:8000</code> . Also <code>https://www.pathpro.tech/api</code> and <code>https://155-138-233-35.sslip.io/api</code> .
Local API	<code>http://127.0.0.1:8000</code>	<code>cd backend && uv run uvicorn app.main:create_app --factory --reload.No /api prefix.</code>
Local SPA	<code>http://127.0.0.1:5173/api</code>	Vite dev proxy rewrites <code>/api/*</code> to the local API.

Paths below are written without the `/api` prefix. FastAPI's generated OpenAPI document is served at `/openapi.json` (production: `https://pathpro.tech/api/openapi.json`). The interactive `/docs` page loads Swagger UI from a CDN, which the production CSP blocks, so use it locally.

Conventions

Response envelope

Every JSON endpoint returns the same envelope (`api/envelope.py`):

```
{ "success": true, "data": { "...": "..." }, "error": null, "model_version": "pp-20260926-1902-f49c0d2" }
```

```
{ "success": false, "data": null,
  "error": { "code": "OUT_OF_COVERAGE", "message": "PathPro covers the City of Atlanta for now. Pick a
starting point and destination inside city limits." },
  "model_version": "pp-20260926-1902-f49c0d2" }
```

- `error.code` is stable and machine-readable; `error.message` is user-facing copy.
- Request-validation failures (bad JSON, wrong types, values outside limits, unknown fields where forbidden) return **422** with code `BAD_REQUEST` and the message "That request was not valid." Validator details are never echoed.
- Unhandled exceptions return **500** INTERNAL "Something went wrong. Please retry." with no stack trace.
- Exceptions: `POST /tts` returns raw `audio/mpeg` on success; unknown paths and wrong methods get FastAPI's default `{"detail": "Not Found"}` (404) and `{"detail": "Method Not Allowed"}` (405).
- Responses are typed with pydantic response models; the SPA validates every body with zod (`frontend/src/api/schemas.ts`).

Time and condition parameters

Parameter	Type	Accepted values
-----------	------	-----------------

<code>t / depart_at</code>	string, max 40 chars	now, <code>+Nm</code> or <code>+Nh</code> (N up to 3 digits), or ISO-8601. Naive ISO is Atlanta wall-clock time. Must be within 400 days of now. Invalid → 422 <code>BAD_TIME</code> (segments, areas, explain) or <code>BAD_DEPARTURE</code> (routes).
<code>cond</code>	enum	<code>live</code> (Open-Meteo forecast for that hour), <code>dry</code> , <code>wet</code> (override). Default <code>live</code> .
<code>mode</code>	enum	<code>walk</code> , <code>bike</code> , <code>ebike</code> , <code>scooter</code> . Default <code>walk</code> . Ride modes need the ride model (else 503 <code>MODE_UNAVAILABLE</code>).

`condition_used` in responses: `{cond: "dry"|"wet", source: "live"|"override"|"assumed", label}`. `assumed` means the forecast was unavailable and dry was used.

Rate limits

Implemented in `app/middleware.py` as a per-client sliding 60-second window. The client key is `request.client.host`, which uvicorn sets from Caddy's `X-Forwarded-For` only because `--forwarded-allow-ips 127.0.0.1` trusts the local proxy; IPv6 addresses are grouped by /64.

Class	Default limit	Env var	Applies to
Exempt	none	n/a	Paths starting <code>/healthz</code> , <code>/static</code>
General	300 requests / min / client	<code>RATE_LIMIT_PER_MINUTE</code>	Every other request, including all reads, <code>POST /routes</code> , and <code>PUT /walks/{id}/position</code>
Paid / write	30 requests / min / client, in addition to the general limit	<code>PAID_RATE_LIMIT_PER_MINUTE</code>	Any method on <code>/explain</code> , <code>/geocode</code> , <code>/tts</code> ; <code>POST /reports</code> ; <code>POST /walks</code>

Over either limit → **429** `RATE_LIMITED` "Too many requests. Try again in a minute." (envelope with `model_version: null`).

Why `PUT /walks/{id}/position` is on the general limit: at the expo every visitor shares the venue's NAT address, and each walker posts about 12 updates per minute. On the paid limit, three walkers would exhaust it for the whole hall. Instead an in-memory per-walk gate rejects updates that arrive within 3 seconds of the last accepted one (429 `WALK_THROTTLED`) before any database read.

Daily budgets (per process, reset at local midnight) cap spend independently of the per-client limits: LLM generations `LLM_DAILY_BUDGET` (3,000), Geoapify calls `GEOCODE_DAILY_BUDGET` (2,500), ElevenLabs calls `TTS_DAILY_BUDGET` (500). Past a budget the endpoint degrades (template text, empty results, device voice) instead of failing.

CORS

`CORSMiddleware` with `allow_origins` from `ALLOWED_ORIGINS` (comma-separated), methods `GET`, `POST`, `PUT`, header `Content-Type`, `allow_credentials=False`. `deploy/deploy.sh` sets the origins to `https://` plus each Caddy site name (`pathpro.tech`, `www.pathpro.tech`, `155-138-233-35.sslip.io`). Preflights from other origins get 400. The SPA itself is same-origin, so CORS matters only to third-party callers.

Endpoint summary

Method	Path	Rate class	Purpose
GET	<code>/healthz</code>	exempt	Liveness and dependency status
GET	<code>/meta</code>	general	Model version, coverage, frames layout, modes, headline metrics
POST	<code>/routes</code>	general	Fastest vs lower-risk route

GET	/segments/{seg_id}	general	Street score, factors, history
GET	/segments/{seg_id}/hourly	general	Crashes by hour from Tiger Data
GET	/segments/{seg_id}/reports	general	Community reports on a street
POST	/explain	paid	One-to-three-sentence grounded explanation
POST	/tts	paid	MP3 of the server's explanation
GET	/geocode	paid	Address autocomplete
GET	/conditions/live	general	Current dry/wet condition
GET	/areas/lookup	general	City Pulse score at a point
GET	/areas/{cell}	general	City Pulse score for an H3 cell
POST	/reports	paid	Flag a traffic-related street issue
GET	/reports	general	Reports in a map viewport
GET	/reports/summary	general	Active reports per category
POST	/walks	paid	Start sharing a walk
PUT	/walks/{walk_id}/position	general + per-walk gate	Owner position update
GET	/walks/{walk_id}	general	What a follower sees
GET	/safety/meta	general	Safety-layer sources and day parts
GET	/safety/hexes	general	Safety hexes in a viewport for a day part
GET	/safety/help-points	general	Help points in a viewport
GET	/transit/stations	general	MARTA rail stations in coverage
GET	/static/{model_version}/{file}	exempt	Bundle files (served by Caddy from disk in production)

Health and metadata

GET /healthz

No parameters. Pings Tiger Data (SELECT 1, 3 s connect, 800 ms statement) and MongoDB Atlas (ping) on every call.

Field	Type	Meaning
status	ok degraded	degraded when database or reports is unavailable
model_version	string	Served walk bundle
segments	int	Walk segments (49,915)
graph_nodes	int	Walk graph nodes (84,758)
database	ok unavailable not_configured	Tiger Data
reports	ok unavailable not_configured	MongoDB Atlas
safety	ok unavailable	Personal-safety layer loaded

modes	{walk: "ok", ride: "ok" "unavailable"}	Ride model loaded
-------	--	-------------------

```
{
  "success": true,
  "data": {
    "status": "ok",
    "model_version": "pp-20260926-1902-f49c0d2",
    "segments": 49915,
    "graph_nodes": 84758,
    "database": "ok",
    "reports": "ok",
    "safety": "ok",
    "modes": {
      "walk": "ok",
      "ride": "ok"
    },
    "error": null,
    "model_version": "pp-20260926-1902-f49c0d2"
  }
}
```

The endpoint always returns 200 while the process is up; read `status` for dependency health.

GET /meta

No parameters.

Field	Type	Meaning
model_version, data_through	string	Bundle id; last crash date (2026-09-19)
n_segments	int	Walk segments
coverage_bbox	[west, south, east, north]	City of Atlanta bounds
day_groups, conditions	string[]	weekday, friday, saturday, sunday; dry, wet
reference_dates	{day_group: date}	Date each frame set was lit for
frame_light	{day_group: [24 × "day" "twilight" "dark"]}	Light per frame hour
static_base	string	/static/{model_version}; frames at {static_base}/frames_{day}_{cond}.bin
headline	object	Walk headline metrics (capture, CI, baselines, ROC-AUC, temporal gain)
spatial_factors, temporal_factors	{key, label, points: 0}[]	Factor keys and labels
modes	{key, label, available, speed_kmh, network: "walk" "ride", static_prefix: "" "ride_"}[]	Travel modes
ride_model	object null	Ride headline metrics

```
{
  "success": true,
  "data": {
    "model_version": "pp-20260926-1902-f49c0d2",
    "data_through": "2026-09-19",
    "n_segments": 49915,
    "coverage_bbox": [-84.55085, 33.64792, -84.28956, 33.88682],
    "day_groups": ["weekday", "friday", "saturday", "sunday"],
    "conditions": ["dry", "wet"],
    "reference_dates": {
      "friday": "2026-09-25",
      "saturday": "2026-09-26",
      "sunday": "2026-09-27",
      "weekday": "2026-09-28"
    },
    "static_base": "/static/pp-20260926-1902-f49c0d2",
    "modes": [
      {
        "key": "walk",
        "label": "Walk",
        "available": true,
        "speed_kmh": 4.68,
        "network": "walk",
        "static_prefix": ""
      },
      {
        "key": "bike",
        "label": "Bike",
        "available": true,
        "speed_kmh": 15.0,
        "network": "ride",
        "static_prefix": "ride_"
      },
      ...
    ],
    "headline": {
      "capture_top10": 0.7433,
      "capture_top10_ci95": [0.7049, 0.7827],
      "roc_auc": 0.889,
      "...": "..."
    },
    "spatial_factors": [
      {
        "key": "history",
        "label": "Pedestrian crash history here",
        "points": 0
      },
      ...
    ],
    "error": null,
    "model_version": "..."
  }
}
```

Routing

POST /routes

Request body (RouteRequest):

Field	Type	Limits / default
origin	{lat, lon}	lat -90..90, lon -180..180; must be inside the City of Atlanta (city hexes)
destination	{lat, lon}	same
depart_at	string	default now; see time parameters
cond	live dry wet	default live
prefer	lower_traffic_risk lit_and_busy	default lower_traffic_risk; lit_and_busy changes walks after dark only and is ignored for ride modes
mode	walk bike ebike scooter	default walk

Processing: coverage check → snap each end to the nearest graph node (≤ 150 m) → resolve condition → plan in a worker thread (see [architecture §6.1](#)) → add community reports on the recommended walk route (≤ 0.5 s wait) → store the result under `route_key` for `/explain`.

Response RoutesData:

Field	Type	Meaning
condition_used	ConditionUsed	Dry/wet actually used
depart_at	ISO string	Resolved departure (Atlanta offset)
fastest	RouteOut	Fastest route, re-scored at traversal times
pathpro	RouteOut null	Lower-risk route, or null when the fastest already is
message_code	ok tradeoff_exists fastest_is_lower_risk long_trip	Drives the copy

message	string null	e.g. "The fastest route is already the lower-risk option."
time_cost_min	float null	Extra minutes for pathpro
exposure_reduction_pct	int null	Percent less expected-crash exposure (never ≤ 0)
unavoidable	string[]	High-risk streets both routes use
avoided	{seg_id, name, score}[]	High-risk streets (≥ 75) on the fastest route that pathpro avoids entirely
route_key	16 hex chars	Key for /explain and /tts
reports	ReportOut[]	Community reports on the recommended route (walk only; context only)
mode, prefer	enums	Mode and the preference actually applied

RouteOut: coords ([lon, lat][], 6 decimals), duration_s, distance_m, risk_score (0–100, length-weighted), band (Lower/Moderate/Elevated/High), exposure (expected-crash exposure: Σ density per 100 m $\times$ length), high_risk_m (metres scoring ≥ 75), limited_data_m, segment_ids, top_segments (up to 3 named streets contributing most exposure), alerts (stretches ≥ 90 merged within 100 m: start_m, end_m, names, score, stretches), safety (walk only, else null: lit_share, busy_share, help_points_within_100m, crimes_persons_nearby, day_part).

Sample (Klaus Building $\rightarrow$ Midtown MARTA, Friday 22:30, cond: "wet"; coordinate and id arrays trimmed):

```
{
  "success": true,
  "data": {
    "condition_used": {
      "cond": "wet",
      "source": "override",
      "label": "Wet (your choice)"
    },
    "depart_at": "2026-09-25T22:30:00-04:00",
    "fastest": {
      "coords": [
        [-84.39626, 33.77705],
        ["...", "duration_s": 1104.9, "distance_m": 1436.3, "risk_score": 98, "band": "High",
        "exposure": 1.9257, "high_risk_m": 1292.0, "limited_data_m": 60.0, "segment_ids": [3723, 3724, "..."],
        "top_segments": [
          {"seg_id": 13343, "name": "Fifth Street Northwest", "score": 99},
          {"seg_id": 14275, "name": "Peachtree Place Northwest", "score": 100}
        ],
        "alerts": [
          {"start_m": 89.0, "end_m": 1372.0, "names": ["Ferst Drive Northwest", "..."], "score": 100, "stretches": 15}
        ],
        "safety": {
          "lit_share": 1.0, "busy_share": 1.0, "help_points_within_100m": 7, "crimes_persons_nearby": 12, "day_part": "night"
        }
      },
    "pathpro": {
      "duration_s": 1355.0, "distance_m": 1761.5, "risk_score": 93, "band": "High", "exposure": 0.8846, "...": "...",
      "message_code": "tradeoff_exists",
      "message": "A lower-risk route exists but adds 12 min. Showing the best option under 6 extra min.",
      "time_cost_min": 4.2, "exposure_reduction_pct": 54, "unavoidable": ["Fifth Street Northwest"],
      "avoided": [
        {"seg_id": 18085, "name": "8th Street Northwest", "score": 97},
        {"seg_id": 14275, "name": "Peachtree Place Northwest", "score": 100}
      ],
      "route_key": "f5ae7c3aebe46e80", "reports": [], "mode": "walk", "prefer": "lower_traffic_risk",
      "error": null, "model_version": "pp-20260926-1902-f49c0d2"
    }
  }
}
```

Errors: 422 BAD_REQUEST, BAD_DEPARTURE, OUT_OF_COVERAGE, SNAP_T00_FAR ("Move the pin closer to a sidewalk or street."), T00_CLOSE (under 60 m), NO_CONNECTION; 503 MODE_UNAVAILABLE; 429 RATE_LIMITED.

Street segments

GET /segments/{seg_id}

Parameter	In	Type	Default
seg_id	path	int ($0 \leq \text{id} < \text{n_segments}$)	required
t	query	string ≤ 40	now
cond	query	live dry wet	live
mode	query	ModeKey	walk (ride ids index the ride model; same ids)

Response SegmentDetail: seg_id, name, road_group (arterial/collector/local), score, band, confidence (high/medium/limited), baseline_points, factors (top 5 {key, label, points}, signed), remainder_points, history (crashes, ped_crashes, dark_share, wet_share, period: "2020-2024", bike_crashes for ride modes else null), condition_used, at, mode. Invariant: $\text{baseline_points} + \sum \text{factors.points} + \text{remainder_points} == \text{score}$.

```
{"success":true,"data":{"seg_id":3722,"name":"Fowler Street Northwest","road_group":"local","score":92,"band":"High","confidence":"medium","baseline_points":56,"factors":[{"key":"nearby_history","label":"Pedestrian crashes on nearby streets","points":19}, {"key":"vehicle_crashes","label":"Vehicle crashes on this street","points":9}, {"key":"intersection","label":"Intersection complexity","points":4}, {"key":"length","label":"Block length","points":-3}, {"key":"light","label":"Darkness","points":2}], "remainder_points":5, "history":{"crashes":5.2,"ped_crashes":0.8,"dark_share":0.068,"wet_share":0.227,"period":"2020-2024","bike_crashes":null}, "condition_used":{"cond":"wet","source":"override","label":"Wet (your choice)","at":"2026-09-25T22:30:00-04:00","mode":"walk"}, "error":null,"model_version":"pp-20260926-1902-f49c0d2"}}
```

Errors: 404 NOT_FOUND; 422 BAD_TIME, BAD_REQUEST; 503 MODE_UNAVAILABLE.

GET /segments/{seg_id}/hourly

mode query (default walk); only walk is supported. Reads crashes_hourly in Tiger Data and returns 24 weighted sums in Atlanta local hours.

```
{"success":true,"data":{"seg_id":14275,"crashes":[0.0,"...",0.25,0.5,0.75,0.5],"ped_crashes":[0.0,"...",0.0,0.0,0.0,0.25],"source":"tiger_data"}, "error":null,"model_version":"pp-20260926-1902-f49c0d2"}}
```

(Arrays have 24 entries; weights are fractional because intersection crashes are split across approaches.) Errors: 404 NOT_FOUND (id out of range or mode not walk); 503 HISTORY_UNAVAILABLE (database unset, down, or slower than 800 ms).

GET /segments/{seg_id}/reports

Active community reports on one walk segment, most-confirmed first (max 50). Response ReportOut[] (see reports). Errors: 404 NOT_FOUND; 503 REPORTS_UNAVAILABLE.

Explanations and voice

POST /explain

Body (ExplainRequest):

Field	Type	Limits
kind	segment route	required
seg_id	int ≥ 0	required for segment
route_key	string $^{\wedge}[\text{0-9a-f}]{16}\$$	required for route; must be in the server's recent-routes cache (512 entries)
t	string ≤ 40	segment only, default now
cond	Condition	segment only, default live
mode	ModeKey	segment only, default walk

Response: {text, source} where source $\in$ groq, groq-20b, gemini, template, cache.

```
{"success":true,"data":{"text":"The PathPro route adds 4.2 min and cuts traffic-risk exposure 54% by avoiding Peachtree Place Northwest and Williams Street Northwest. Both routes use Fifth Street Northwest, so stay alert there.","source":"template"},  
"error":null,"model_version":"pp-20260926-1902-f49c0d2"}
```

Guarantees: the text comes only from server-built evidence and passes the validator (≤ 3 sentences, ≤ 420 chars, every number and clock time present in the evidence, no banned framing); otherwise it is the deterministic template. Total LLM time budget 4 s (EXPLAIN_BUDGET_S), first provider 1.6 s (GROQ_BUDGET_S). Errors: 404 ROUTE_EXPIRED, NOT_FOUND; 422 BAD_REQUEST, BAD_TIME; 503 MODE_UNAVAILABLE; 429.

POST /tts

Same body as /explain. The server produces (or reuses) its own explanation and sends that text to ElevenLabs (eleven_flash_v2_5, mp3_44100_64, 6 s timeout). Clients cannot supply text.

- 200: audio/mpeg body, Cache-Control: no-store.
- 503 TTS_UNAVAILABLE "Voice is unavailable; using the device voice." (no key or voice id, text over 420 chars, daily budget spent, or upstream error). The SPA then uses the browser's speech synthesis.

Places and conditions

GET /geocode

Parameter	Type	Limits
q	string	1–80 chars (whitespace collapsed; fewer than 3 characters returns [])

Geoapify autocomplete filtered to the city rectangle and biased toward Georgia Tech, max 5 results, cached 1 h. The key stays on the server. Response {label, address, lat, lon, in_coverage}[] . Failures, missing key, or an exhausted budget return [] .

GET /conditions/live

Response ConditionUsed for the current hour: {"cond":"dry","source":"live","label":"Dry · live forecast"}. Forecast cached 10 min; stale forecast used up to 1 h; otherwise {"cond":"dry","source":"assumed","label":"Live weather unavailable – using dry conditions."}. Freezing-precipitation and snow weather codes count as wet with the label "Icy conditions are rarer in our data — use extra caution."

City Pulse

GET /areas/lookup

Parameter	Type	Limits
lat, lon	float	−90..90, −180..180
t, cond	as above	defaults now, live

GET /areas/{cell}

cell must match `^[0-9a-f]{15}$` (H3 resolution 9); t, cond as above.

Response AreaDetail: cell, lat, lon, score, band, confidence, baseline_points, factors (top 4, including time_conditions), remainder_points, crashes, ped_crashes, period, in_street_coverage, condition_used, at.

```
{"success":true,"data":
{"cell":"8944c1aaa2fffff","lat":33.754171,"lon":-84.41506,"score":95,"band":"High","confidence":"high",
"baseline_points":57,"factors":[{"key":"vehicle_crashes","label":"Vehicle crashes in this
area","points":30},
{"key":"activity","label":"Pedestrian activity and transit","points":5},
{"key":"nearby","label":"Pedestrian crashes in surrounding areas","points":2},
{"key":"time_conditions","label":"Time of day and
conditions","points":0}],
"remainder_points":1,"crashes":115.0,"ped_crashes":1.0,
"period":"2020-2024","in_street_coverage":true,"condition_used":
{"cond":"wet","source":"override","label":"Wet (your choice)"},
"at":"2026-09-25T22:30:00-04:00"},"error":null,"model_version":"pp-20260926-1902-f49c0d2"}
```

Errors: 404 OUTSIDE_CITY; 422 BAD_TIME, BAD_REQUEST; 503 CITY_PULSE_UNAVAILABLE.

Community street reports (MongoDB Atlas)

Reports are context only: they never change a score, a route choice, or explanation evidence.

POST /reports

Body (extra fields forbidden): {seg_id: int ≥ 0, category} where category ∈ sidewalk_blocked, signal_out, construction, poor_lighting, flooding, fast_traffic. There is no free-text field and no client location: the street name and point come from the bundle's walk graph (midpoint of the segment's first walk edge). One atomic upsert on the unique (seg_id, category) index: a repeat report increments confirmations and extends expiry to 14 days from now.

Response ReportOut: seg_id, category, label, street, lon, lat, confirmations, updated_at, expires_at.
Errors: 404 NOT_FOUND; 422 BAD_REQUEST; 503 REPORTS_UNAVAILABLE; 429.

GET /reports

bbox query: minLon,minLat,maxLon,maxLat, ≤ 120 chars, finite, ordered, each span ≤ 0.3°. Wider or malformed boxes are refused with 422 BAD_BBOX (not truncated). Returns up to 300 active reports (\$geoWithin on the 2dsphere index), most-confirmed first.

GET /reports/summary

Aggregation per category over active reports: {category, label, reports, confirmations}[], sorted by confirmations. Live sample: {"success":true,"data":[],"error":null,...}.

Share my walk (MongoDB Atlas)

POST /walks → 201

Body (extra fields forbidden):

Field	Type	Limits
destination	{label, lat, lon}	label 1–120 chars (trimmed); coordinates finite
eta_s	int	0–43,200
route	[lon, lat][] null	≤ 2,000 points

Destination and every route point must fall in a metro-Atlanta box (−84.85..−83.95, 33.35..34.25) or 422 OUTSIDE_AREA.

Response CreatedWalkOut: walk_id (128 random bits, URL-friendly base64), owner_token (256 random bits; **returned only here**; the server stores its SHA-256), follow_path (/follow/{walk_id}), expires_at (now + 6 h).

PUT /walks/{walk_id}/position

walk_id ≤ 128 chars and must match [A-Za-z0-9_-]{16,64} (else 404). Body (extra fields forbidden): owner_token (16–128 chars), lat, lon (finite, metro box), accuracy_m (0–10,000, optional), eta_s (0–43,200, optional), status (walking|arrived|ended, optional).

Rules: gate check (one accepted update per 3 s per walk, status changes exempt) → load → constant-time token-hash comparison → ended walks stay ended (summary returned unchanged) → area check → optimistic-concurrency write conditioned on the previous updated_at → expires_at moves to now + 6 h.

Response WalkSummaryOut: walk_id, status, destination, eta_s (seconds left; 0 unless walking), eta_at, position (lat, lon, accuracy_m, at) or null, updated_at, expires_at. Never includes the token or its hash.

Errors: 403 WALK_FORBIDDEN; 404 WALK_NOT_FOUND; 422 OUTSIDE_AREA, BAD_REQUEST; 429 WALK_THROTTLED (gate, per-walk interval, or a concurrent write); 503 WALKS_UNAVAILABLE.

GET /walks/{walk_id}

What a follower sees: WalkSummaryOut plus route. Expired walks read as 404 even before the TTL monitor deletes them.

```
{"success":false,"data":null,"error":{"code":"WALK_NOT_FOUND","message":"This shared walk has ended or the link has expired."},"model_version":"pp-20260926-1902-f49c0d2"}
```

Personal-safety layer

All three return 503 `SAFETY_UNAVAILABLE` when the bundle has no safety files. Crime data here is informational; it is never an input to routing or the traffic model.

GET `/safety/meta`

`{data_through, sources: {name, url, license}[], crime_categories: string[], day_parts: {key, label, hours}[]}`.

GET `/safety/hexes`

Parameter	Type	Limits
<code>bbox</code>	string	as <code>/reports</code> ($\leq 0.3^\circ$ span)
<code>hour</code>	int	0–23, optional (default: current Atlanta hour); selects the day part

Response `SafetyHexOut[]` (hexes whose centers are within the box padded by 0.003° , max 4,000): `h3`, `lat`, `lon`, `crimes_persons_12mo`, `crime_band` (lower/typical/higher), `lit_share` (null when under half of walkable length is known), `activity_band` (quiet/moderate/busy/null), `help_points`.

```
{"success":true,"data":
[{"h3":"8944c1a8123ffff","lat":33.756347,"lon":-84.392019,"crimes_persons_12mo":11,"crime_band":"higher",
"lit_share":1.0,"activity_band":"busy","help_points":0}, "..."],"error":null,"model_version":"..."}
```

GET `/safety/help-points`

`bbox` as above. Response `{kind: "blue_light"|"police"|"fire"|"hospital"|"marta", name, lat, lon}[]` (max 1,000).

Transit

GET `/transit/stations`

MARTA heavy-rail stations inside the coverage bounding box (28 of the 38 in `backend/app/domain/marta_stations.json`): `{name, lat, lon, lines}[]`. Used for the MARTA hand-off card on walks longer than 25 minutes.

Static bundle files

GET `/static/{model_version}/{file}`. In production Caddy serves `/srv/pathpulse/artifacts/{version}/{file}` directly with `Cache-Control: public, max-age=31536000, immutable`; the API also mounts the same directory. Any version still on disk is reachable, which is why version-pinned caching can use an immutable header. File formats are listed in [data_and_models.md §11](#).

Error code index

Code	HTTP	Raised by
------	------	-----------

BAD_REQUEST	422	Request validation; /explain without seg_id
BAD_TIME / BAD_DEPARTURE	422	Unparseable or out-of-range time
BAD_BB0X	422	/reports, /safety/hexes, /safety/help-points
OUT_OF_COVERAGE	422	/routes
SNAP_TOO_FAR	422	/routes (> 150 m from the graph)
TOO_CLOSE	422	/routes (< 60 m)
NO_CONNECTION	422	/routes
OUTSIDE_AREA	422	/walks*
WALK_FORBIDDEN	403	Wrong owner token
NOT_FOUND	404	Unknown segment
OUTSIDE_CITY	404	/areas*
ROUTE_EXPIRED	404	/explain, /tts for a route not in cache
WALK_NOT_FOUND	404	Unknown or expired walk
RATE_LIMITED	429	Middleware
WALK_THROTTLED	429	Share-my-walk update gate or race
INTERNAL	500	Unhandled exception
MODE_UNAVAILABLE	503	Ride mode without a ride model
HISTORY_UNAVAILABLE	503	Tiger Data
REPORTS_UNAVAILABLE	503	MongoDB Atlas (reports)
WALKS_UNAVAILABLE	503	MongoDB Atlas (walks)
SAFETY_UNAVAILABLE	503	No safety layer
CITY_PULSE_UNAVAILABLE	503	No City Pulse files
TTS_UNAVAILABLE	503	ElevenLabs

Demo-only codes returned by the browser's offline transport (never by the server): DEMO_ONLY, and the 503-style codes above for features the demo hides.

Deployment and operations

How PathPro is hosted, configured, deployed, observed, and rolled back. Sources: `deploy/*.sh`, `deploy/Caddyfile`, `deploy/Caddyfile.local`, `deploy/pathpulse.service`, `backend/app/config.py`, `.env.example`, `docs/sponsor_checklist.md`. Internal identifiers (`pathpulse.service`, `/srv/pathpulse`, the `pathpulse` system user, database names, `~/.ssh/pathpulse_ed25519`) keep the project's original name on purpose; the user-facing brand is PathPro.

1. Environments

Environment	How it runs	Used for
Local development	<code>cd backend && uv run uvicorn app.main:create_app --factory --reload (port 8000) and cd frontend && npm run dev</code> (Vite on 5173, proxies <code>/api</code> and <code>/static</code> to 8000)	Day-to-day work; Playwright starts both via <code>playwright.config.ts</code>
Local production-like + tunnel (fallback, retired)	<code>deploy/run_live.sh</code> : builds the SPA, starts uvicorn, Caddy with <code>deploy/Caddyfile.local</code> on <code>:8080</code> , and a Cloudflare quick tunnel; <code>deploy/tunnel_watchdog.sh</code> restarts the tunnel every 60 s if <code>/api/healthz</code> fails twice	Friday-night hosting before the VM; kept as a runbook fallback (section 10)
Production	One Vultr VM, Caddy + systemd, <code>deploy/go.sh pathpro.tech</code>	<code>https://pathpro.tech</code>
Offline demo	Any build, URL <code>?demo=1</code>	Expo table; works with the network off

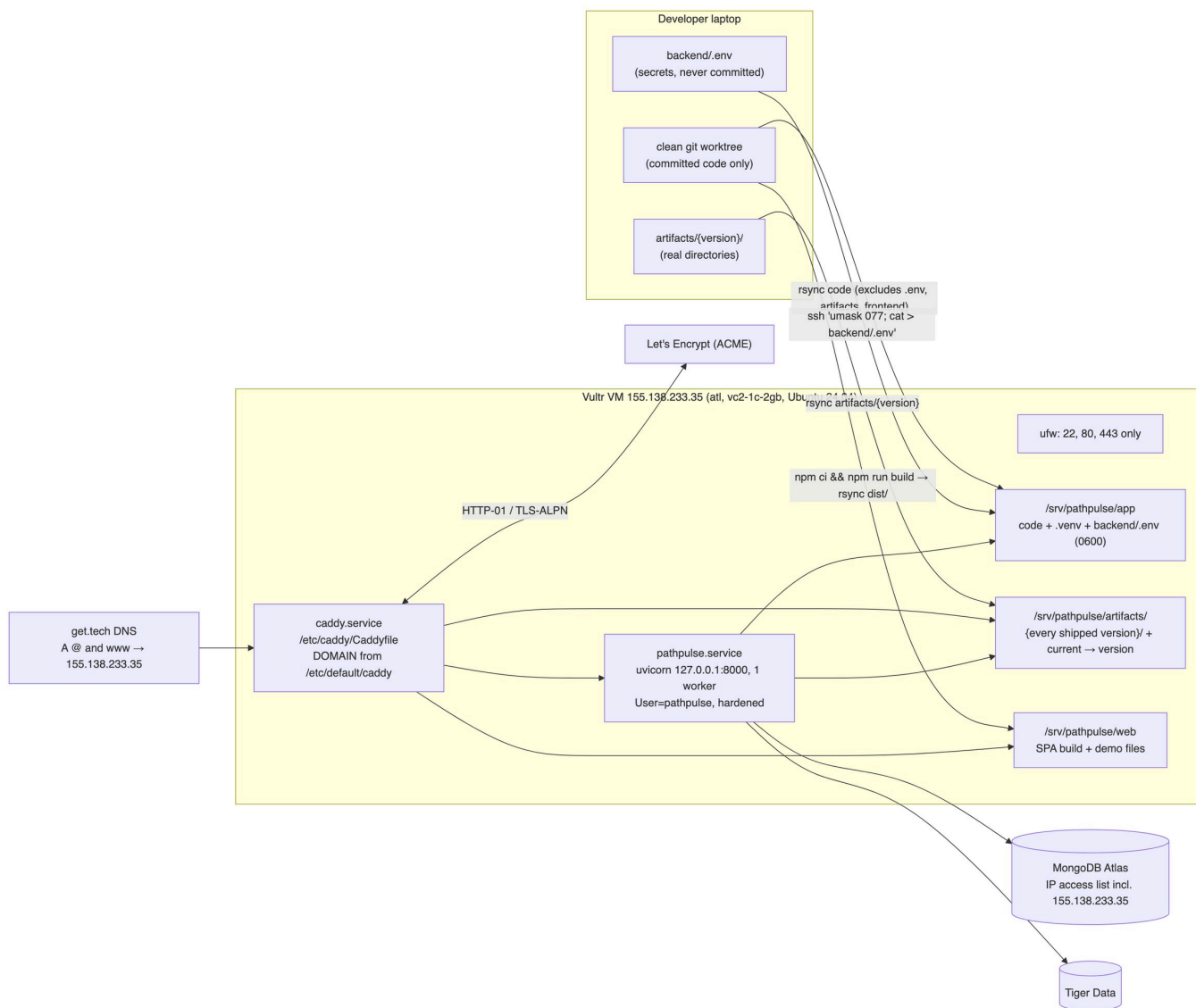


Figure O1. Production deployment topology.

2. Configuration

2.1 Backend environment variables

Read by pydantic-settings from the process environment or `backend/.env` (`app/config.py`). Secrets are typed `SecretStr` and never logged. Names only; values live in `backend/.env` on the laptop and on the VM.

Name	Default	Purpose
APP_ENV	development	Set to <code>production</code> by the unit and <code>deploy.sh</code> ; informational
ARTIFACTS_DIR	<repo>/artifacts/current	Bundle directory; production <code>/srv/pathpulse/artifacts/current</code>
ALLOWED_ORIGINS	http://localhost:5173	CORS origins, comma-separated; <code>deploy.sh</code> rewrites it
RATE_LIMIT_PER_MINUTE	300	General per-client limit

PAID_RATE_LIMIT_PER_MINUTE	30	Paid/write per-client limit
LLM_DAILY_BUDGET	3000	LLM generations per day (process)
GEOCODE_DAILY_BUDGET	2500	Geoapify calls per day
TTS_DAILY_BUDGET	500	ElevenLabs calls per day
EXPLAIN_BUDGET_S	4.0	Total explanation time budget
GROQ_BUDGET_S	1.6	First provider's share of that budget
GROQ_API_KEY	none	Groq (secret)
GROQ_MODEL / GROQ_FALLBACK_MODEL	openai/gpt-oss-120b / openai/gpt-oss-20b	Groq models
GEMINI_API_KEY	none	Gemini (secret)
GEMINI_MODEL	gemini-3.8-flash	Gemini model
ELEVENLABS_API_KEY	none	ElevenLabs (secret)
ELEVENLABS_VOICE_ID	none	Voice to use
ELEVENLABS_MODEL	eleven_flash_v2_5	TTS model
GEOAPIFY_API_KEY	none	Geoapify (secret)
DATABASE_URL	none	Tiger Data Postgres URL (secret)
MONGODB_URI	none	Atlas connection string (secret)
MONGODB_DB	pathpulse	Atlas database name
VULTR_API_KEY	none	Read only by <code>deploy/provision_vultr.sh</code> on the laptop; not used by the app

Every secret is optional: without it the matching feature degrades (see [architecture §9](#)).

2.2 Frontend build variables

Name	Effect
VITE_BASE	Base path (default <code>/</code>)
VITE_FORCE_DEMO=1	Fixtures only, no backend (used by the retired static build)
VITE_LIVE_CONFIG=1	Reads <code>live.json</code> for a remote API origin and health-checks it, else falls back to the demo (retired static build)
PATHPULSE_API	Dev-server proxy target (default <code>http://127.0.0.1:8000</code>)

The production build uses none of these: the SPA and API share one origin.

3. Provisioning (`deploy/go.sh`)

`deploy/go.sh <domain>` runs six steps and can be re-run: each step reuses what already exists.

Step	Script	What it does
------	--------	--------------

1	<code>app.tools.check_keys</code>	One minimal live call per configured key (Groq models list, Gemini model, Geoapify search, ElevenLabs voice, Tiger extensions, Atlas ping and indexes). Prints <code>OK</code> / <code>MISSING</code> / <code>FAIL</code> and never the key. Any <code>FAIL</code> stops the deploy.
2	<code>deploy/provision_vultr.sh</code>	Skipped if <code>deploy/.host</code> exists. Otherwise, with <code>VULTR_API_KEY</code> : reuse a VM labelled <code>pathpulse</code> or create one (<code>region atl</code> , <code>plan vc2-1c-2gb</code> , Ubuntu 24.04 x64, backups disabled) with a new <code>ed25519</code> key <code>~/.ssh/pathpulse_ed25519</code> ; wait for <code>ok</code> and write the IP to <code>deploy/.host</code> (gitignored).
3	<code>deploy/bootsrap.sh</code> (over SSH as root)	Installs Caddy from the Cloudsmith apt repo, <code>rsync</code> , <code>ufw</code> , <code>python3.12</code> ; creates the <code>pathpulse</code> system user and <code>/srv/pathpulse/{app,artifacts,web}</code> ; installs <code>uv</code> for that user; writes <code>DOMAIN=<site list></code> to <code>/etc/default/caddy</code> with a drop-in so Caddy reads it; <code>ufw</code> allows OpenSSH, 80, 443 and is enabled.
4	<code>deploy/deploy.sh</code>	Build and ship (section 4).
5	<code>db.load_tiger</code>	Only when <code>LOAD_TIGER=1</code> and <code>DATABASE_URL</code> is set: loads schema, segments, crashes, and the risk grid (multi-minute, one-time).
6	Smoke test	Prints the DNS answer for the domain and <code>curls https://<domain>/api/healthz</code> ; the <code>ssllip.io</code> name is always reachable.

The Caddy site list is "`<domain>`, `www.<domain>`, `<ip-with-dashes>.ssllip.io`". The `ssllip.io` name gives real HTTPS before DNS for the domain propagates.

4. Deploying a new version (`deploy/deploy.sh`)

```
RSYNC_RSH="ssh -i ~/.ssh/pathpulse_ed25519" \
  bash deploy/deploy.sh root@155.138.233.35 "pathpro.tech, www.pathpro.tech, 155-138-233-35.ssllip.io"
```

What it does, in order:

1. `VERSION` = the target of the local `artifacts/current` symlink.
2. `npm ci && npm run build` in `frontend/` (type-check, then Vite build into `frontend/dist/`).
3. `rsync -az --delete` the repository to `/srv/pathpulse/app/`, excluding `.git`, `node_modules`, `.venv`, `cache`, `data/raw`, `data/interim`, `artifacts`, `frontend`, `backend/.env`, `__pycache__`.
4. `rsync -az artifacts/$VERSION` to `/srv/pathpulse/artifacts/` (**no** `--delete`, so earlier versions stay on the server).
5. `rsync -az --delete frontend/dist/` to `/srv/pathpulse/web/`.
6. Stream `backend/.env` over SSH with `umask 077` (macOS `rsync` has no `--chmod`).
7. On the VM: point `/srv/pathpulse/artifacts/current` at the new version; remove and re-append `ALLOWED_ORIGINS`, `ARTIFACTS_DIR`, `APP_ENV` so production values always win; `chmod 600 backend/.env`; `chown -R pathpulse`; `uv sync --package pathpulse-backend --frozen --no-dev --python /usr/bin/python3.12 --python-preference only-system`; install the unit and Caddyfile; `systemctl enable --now`, `restart pathpulse`, `reload caddy`; poll `http://127.0.0.1:8000/healthz` for up to 30 s.
8. From the laptop, `curl https://<each origin>/api/healthz`.

4.1 Clean-worktree procedure (used since the ride release)

Deploying from the working copy risks shipping uncommitted edits made by other people or agents. Deploys therefore run from a clean git worktree of the commit being released:

```
git worktree add <scratch>/wt-deploy <commit> # committed code only
cp -R artifacts/pp-20260926-1902-f49c0d2 <scratch>/wt-deploy/artifacts/ # the REAL directory
ln -sfn pp-20260926-1902-f49c0d2 <scratch>/wt-deploy/artifacts/current
cp -R frontend/public/demo/static <scratch>/wt-deploy/frontend/public/demo/ # gitignored demo files
cd <scratch>/wt-deploy && RSYNC_RSH="ssh -i ~/.ssh/pathpulse_ed25519" \
  bash deploy/deploy.sh root@155.138.233.35 "pathpro.tech, www.pathpro.tech, 155-138-233-35.sslip.io"
```

- **Copy the bundle directory, do not symlink it.** `artifacts/` is gitignored, so a fresh worktree has none. `deploy.sh` ships it with `rsync -a`, which copies a symlink as a symlink; a symlinked bundle would arrive on the VM as a dangling link and the API would fail its startup bundle check.
- `backend/.env` is read from the worktree path; copy it in (it is never committed) or the deploy leaves the server's existing file in place except for the three production lines.
- The demo geometry under `frontend/public/demo/static/` is gitignored and regenerated by `record_demo`; copy it so `?demo=1` keeps its map on the live site.

5. Caddy and systemd hardening

Caddy (deploy/Caddyfile):

Route	Behaviour
<code>/api/*</code>	<code>handle_path</code> strips <code>/api</code> , <code>reverse_proxy 127.0.0.1:8000</code> ; Caddy sets <code>X-Forwarded-For</code> to the real client
<code>/static/*</code>	Files from <code>/srv/pathpulse/artifacts</code> , <code>Cache-Control: public, max-age=31536000, immutable</code>
<code>/demo/*</code>	Files from <code>/srv/pathpulse/web</code> , <code>max-age=3600</code>
everything else	SPA with <code>try_files {path} /index.html</code>

Headers on every response: `Strict-Transport-Security: max-age=31536000`; `includeSubDomains`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: camera=(), microphone=(), geolocation=(self)`, a `Content-Security-Policy` allowing only self plus Google Fonts and OpenFreeMap tiles (full text in [security_privacy.md](#)), and `Server` removed. Responses are compressed with `zstd` or `gzip`. TLS certificates are issued and renewed automatically.

systemd (deploy/pathpulse.service):

Setting	Why
<code>User=pathpulse, Group=pathpulse</code>	Unprivileged system user
<code>ExecStart=.venv/bin/uvicorn app.main:create_app --factory --host 127.0.0.1 --port 8000 --workers 1 --proxy-headers --forwarded-allow-ips 127.0.0.1</code>	Loopback only; trust forwarded headers only from local Caddy; one worker because caches, limits, and the walk gate are in memory
<code>EnvironmentFile=/srv/pathpulse/app/backend/.env, Environment=APP_ENV=production, ARTIFACTS_DIR=/srv/pathpulse/artifacts/current</code>	Config
<code>Environment=HOME=/srv/pathpulse</code>	<code>libpq</code> probes <code>~/.postgresql</code> for client certificates. With <code>ProtectHome=true</code> , <code>/home</code> is unreadable and that probe fails the Tiger Data connection. A <code>HOME</code> outside <code>/home</code> keeps the hardening and lets Tiger connect.

<code>NoNewPrivileges</code> , <code>PrivateTmp</code> , <code>ProtectSystem=strict</code> , <code>ProtectHome=true</code> , <code>ReadOnlyPaths=/srv/pathpulse</code> , <code>empty</code> <code>CapabilityBoundingSet</code> , <code>LockPersonality</code> , <code>RestrictSUIDSGID</code>	The API only reads its code and artifacts
<code>Restart=always</code> , <code>RestartSec=2</code>	Self-healing

Because of `ProtectHome=true`, the venv must use the system Python (`/usr/bin/python3.12`), not a uv-managed interpreter under `/home` (fix 47fc47e).

6. DNS, external services, and secrets

Item	Setting
Domain	<code>pathpro.tech</code> , registered at get.tech with the MLH .tech code (valid to 2027-09-26)
DNS	A records <code>@</code> and <code>www</code> → 155.138.233.35
TLS	Let's Encrypt via Caddy for <code>pathpro.tech</code> , <code>www.pathpro.tech</code> , <code>155-138-233-35.sslip.io</code>
MongoDB Atlas	Free M0 cluster (AWS us-east-1). Network Access list must include 155.138.233.35 , or <code>/healthz</code> shows reports: <code>unavailable</code> and reports and sharing turn off. TLS verified with certifi's CA bundle. Indexes are created at API startup.
Tiger Data	Free shared service (AWS us-east-1). Connection string in <code>DATABASE_URL</code> . Needs the <code>timescaledb</code> and <code>postgis</code> extensions (checked by <code>check_keys</code>).
Secrets	Only in <code>backend/.env</code> (laptop and VM, mode 0600 on the VM). <code>.gitignore</code> excludes <code>.env</code> and <code>.env.*</code> except <code>.env.example</code> , which holds names only. Keys never reach the browser: the SPA calls the API, which calls the providers.
Key capture helpers	<code>deploy/capture_keys.py</code> (clipboard watcher) and <code>deploy/chrome_keys.py</code> (browser helper) write keys straight into <code>backend/.env</code> , recognize them by format, and mask any key-like string in their own output.

7. Health checks and logs

Check	Command	Healthy result
Public health	<code>curl -s https://pathpro.tech/api/healthz</code>	<code>status: ok</code> , <code>database: ok</code> , <code>reports: ok</code> , <code>safety: ok</code> , <code>modes.ride: ok</code> , <code>expected model_version</code>
Local health on the VM	<code>curl -s http://127.0.0.1:8000/healthz</code>	same
Keys	<code>cd backend && uv run python -m app.tools.check_keys</code>	all OK
Model served	<code>curl -s https://pathpro.tech/api/meta</code>	<code>model_version</code> , <code>n_segments: 49915</code> , four modes available
Headers	<code>curl -sI https://pathpro.tech/</code>	HSTS, CSP, X-Frame-Options: DENY

Logs:

- API: `journalctl -u pathpulse -f`. Startup logs PathPro API ready: model <version>, <n> walk segments, ride <n|unavailable>. Provider, database, and Atlas failures log the exception **type** only, because exception text can echo a connection string. `htpx` is set to WARNING so the Geoapify key (a query parameter) never lands in logs.
- Caddy: JSON access log at `/var/log/caddy/pathpulse.log`; service log in `journalctl -u caddy`.

8. Rollback

Every shipped bundle stays in `/srv/pathpulse/artifacts/` (step 4 has no `--delete`), and bundle versions are immutable. Each version also has distinct `/static/<version>/` URLs, so browsers never mix frames from two versions.

Model rollback (no code change):

```
ssh -i ~/.ssh/pathpulse_ed25519 root@155.138.233.35 \  
  'ls /srv/pathpulse/artifacts && ln -sf /srv/pathpulse/artifacts/<older-version>  
/srv/pathpulse/artifacts/current && systemctl restart pathpulse'  
curl -s https://pathpro.tech/api/healthz # model_version should now be <older-version>
```

A bundle without ride or safety files is still valid: ride modes and the safety layer switch off; walking keeps working. Code rollback: create a worktree at the previous commit and run the procedure in section 4.1 (the API and a bundle must be compatible; the loader treats ride, City Pulse, and safety files as optional).

9. Cost

Item	Cost
Vultr <code>vc2-1c-2gb</code> in Atlanta	about \$10 per month, paid from a \$100 MLH Vultr credit (expires 2026-10-27)
pathpro.tech	Free for the first year with the MLH .tech code
MongoDB Atlas M0, Tiger Data free shared service	Free tiers
Groq, Gemini, ElevenLabs, Geoapify	Free tiers, bounded by the daily budgets in section 2.1
OpenFreeMap tiles, Open-Meteo	Free, no key

10. Runbook

Symptom	Likely cause	Action
<code>/api/*</code> returns 502	API not running (bad bundle, crash loop)	<code>journalctl -u pathpulse -n 100</code> ; a <code>BundleError</code> means a missing file or checksum mismatch: re-ship the bundle directory or roll back (section 8)
<code>/healthz</code> reports: <code>unavailable</code>	Atlas unreachable or VM IP not in the access list	Check Atlas Network Access; the API retries after a 30 s cooldown
<code>/healthz</code> database: <code>unavailable</code>	Tiger Data down or slow; <code>HOME</code> missing from the unit	Confirm <code>Environment=HOME=/srv/pathpulse</code> ; run <code>check_keys</code>
Explanations always source: <code>"template"</code>	No LLM keys, provider errors, or daily budget spent	<code>check_keys</code> ; look for <code>explain provider ... failed</code> or <code>output rejected</code> in the journal
"Listen" uses the device voice	ElevenLabs key, voice id, or budget	<code>check_keys</code> ; 503 <code>TTS_UNAVAILABLE</code> is expected fallback behaviour
Many 429s from one venue	Shared NAT address	Raise <code>RATE_LIMIT_PER_MINUTE</code> in <code>backend/.env</code> and restart (paid limits stay tight)
Certificate errors on the domain	DNS not pointing at the VM yet	Use <code>https://155-138-233-35.sslip.io</code> ; check <code>dig +short pathpro.tech</code>

VM unreachable	Provider outage	Fallback: <code>deploy/run_live.sh</code> on a laptop serves the same build through a Cloudflare quick tunnel (URL printed and saved to <code>deploy/.tunnel_url</code>); <code>deploy/tunnel_watchdog.sh</code> keeps it alive. For the expo table, <code>?demo=1</code> needs no backend at all.
Need to rotate a key	Exposure or provider revocation	Replace it in <code>backend/.env</code> , re-run <code>deploy.sh</code> (it re-streams the file) or edit on the VM and <code>systemctl restart pathpulse</code> ; revoke the old key at the provider

Security, privacy, and responsible AI

What PathPro protects, how, and where the controls live in code. Everything below was checked against the code and the live headers of <https://pathpro.tech> on 2026-09-26.

1. Threat model summary

PathPro has no user accounts, no payments, and no personal profiles. The assets worth protecting are small but real:

Asset	Threat	Main controls
Sponsor API keys (LLM, voice, geocoding, databases, Vultr)	Leak through the browser, git, logs, or terminal output	Keys only in <code>backend/.env</code> ; server-side proxying; <code>SecretStr</code> ; type-only error logging; gitleaks pre-commit hook; masked key helpers (section 8)
Paid quotas	Cost abuse by scripted clients	Paid rate-limit class, daily budgets, caching, single-flight LLM calls (section 5)
A walker's live location (Share my walk)	Hijacking or reading someone else's walk	128-bit unguessable walk ids, 256-bit owner tokens stored only as SHA-256, constant-time comparison, 6 h TTL, metro-area bounds (section 3)
Users' trust in the numbers	Hallucinated or alarming explanations	Evidence-only LLM input, validator, template fallback, banned framings (section 7)
Neighbourhoods and the people in them	Stigmatizing areas, demographic proxies, crime-driven routing	No demographic, income, or crime features in any model; crime is display-only with a fairness note; tested (section 9)
Service availability	Floods, slow dependencies	Rate limits, timeouts on every call, cooldowns, graceful degradation, a fully offline demo
Server	Remote code execution, lateral movement	Loopback-only API behind Caddy, hardened systemd unit, ufw, no shell-outs or dynamic SQL on the request path (section 6)

Out of scope for a hackathon deployment: DDoS protection beyond the per-client limits, WAF, and multi-region failover.

2. Data minimization

Data	What is kept	What is never kept
Crash records (pipeline)	Location, timestamp, year, pedestrian/cyclist flags, severity letter, reported light and surface, road names	Names, ages, narratives, map URLs (<code>PII_COLUMNS</code> in <code>ingest/clean.py</code> are dropped or never selected)
Reported crimes (safety layer)	Per H3 res-9 hex × day part counts and bands only	Addresses, report numbers, victim fields, points. The fetch requests only <code>OccurredFromDate</code> , <code>NIBRS_Offense</code> , <code>LocationType</code> and geometry; residences, apartments, jails, and shelters are excluded; sex offenses are not included
Routing requests	Nothing persisted. The last 512 route results sit in process memory for <code>/explain</code>	Origin and destination are not written to any database
Learned routines	Browser <code>localStorage</code> only (<code>pathpro:routines:v1</code> , max 200 trips), zod-validated, one tap clears	Nothing is sent to the server

Share-my-walk session in the browser	<code>sessionStorage</code> only (this tab), never <code>localStorage</code> or the URL	
Shared walks (Atlas)	Destination label and point, ETA, latest position and accuracy, optional route, status, timestamps, SHA-256 of the owner token	The token itself; any user identity. The TTL index deletes the document 6 h after the last update
Community reports (Atlas)	Segment id, one of six fixed categories, street name and point from the bundle , confirmation count, timestamps	Free text, reporter identity, the reporter's location. TTL removes a report 14 days after its last confirmation
Analytics	None: no analytics or tracking scripts in the SPA, no cookies set by PathPro	

Known gap: Caddy's JSON access log (`/var/log/caddy/pathpulse.log`) records client IP, user agent, and full request URLs, including query strings such as `/api/areas/lookup?lat=...&lon=...`, `/api/geocode?q=...`, and map viewport boxes. Request bodies (route origins and destinations, walk positions) are not logged. The PRD's NFR-09 asked for coordinates in logs to be rounded to about 100 m; that rounding is not implemented. The log lives only on the VM.

3. Input validation

Every request body and parameter is a pydantic model or a typed FastAPI parameter; failures become 422 `BAD_REQUEST` without echoing validator details.

Surface	Validation
Coordinates	<code>lat</code> $\in [-90, 90]$, <code>lon</code> $\in [-180, 180]$; Share-my-walk also rejects NaN/Infinity and anything outside a metro-Atlanta box
Times	<code>t/depart_at</code> ≤ 40 chars; <code>now</code> , <code>+Nm</code> , <code>+Nh</code> (≤ 3 digits) or ISO-8601 within 400 days
Enums	<code>cond</code> , <code>mode</code> , <code>prefer</code> , <code>kind</code> , <code>report category</code> , <code>walk status</code> are <code>Literal</code> types
Ids and keys	<code>seg_id</code> ≥ 0 and range-checked against the bundle; <code>route_key</code> <code>^[0-9a-f]{16}\$</code> ; H3 <code>cell</code> <code>^[0-9a-f]{15}\$</code> ; <code>walk_id</code> ≤ 128 chars and <code>[A-Za-z0-9_-]{16,64}</code> before any lookup; owner token 16–128 chars
Viewports	<code>bbox</code> ≤ 120 chars, four finite ordered numbers, each span $\leq 0.3^\circ$; wider boxes are refused rather than truncated
Strings	Geocode <code>q</code> 1–80 chars; walk destination label 1–120 chars (trimmed); route $\leq 2,000$ points; ETA ≤ 12 h
Unknown fields	<code>extra="forbid"</code> on report and walk request models
Server-derived data	Report location and street name come from the bundle, never the client; <code>/tts</code> speaks only server-generated text
Untrusted upstream data	Atlas documents are re-validated on read (<code>parse_report</code> , <code>parse_walk</code>) and malformed ones skipped; the SPA zod-validates every API response and every browser-storage read
Output shape	Separate response models, so tokens and hashes cannot leak into walk responses

Database access uses parameterized queries (psycopg `%s` placeholders in `repositories/history.py`) and structured Mongo filters; no query is built by string concatenation from user input.

4. Authentication and authorization

There are no accounts. The only authorization decision is "may this client update this shared walk?":

- `walk_id = secrets.token_urlsafe(16)` (128 bits). Anyone with the follow link can **read** the walk; ids are not enumerable.
- `owner_token = secrets.token_urlsafe(32)` (256 bits), returned once at creation; the database stores `sha256(token)`; comparison uses `hmac.compare_digest`.
- Updates are conditional on the previous `updated_at` (optimistic concurrency), so racing writes cannot overwrite each other.
- The in-memory update gate records only authenticated, saved updates, so a follower who knows the walk id cannot throttle the walker.

5. Rate limiting and cost control

Control	Setting	Code
General limit	300 requests / min / client (<code>RATE_LIMIT_PER_MINUTE</code>)	<code>middleware.py</code>
Paid/write limit	30 / min / client on <code>/explain</code> , <code>/geocode</code> , <code>/tts</code> , <code>POST /reports</code> , <code>POST /walks</code> (<code>PAID_RATE_LIMIT_PER_MINUTE</code>)	same
Client identity	<code>request.client.host</code> set by uvicorn from <code>X-Forwarded-For</code> only for the trusted local proxy (<code>--forwarded-allow-ips 127.0.0.1</code>); raw headers from the internet cannot spoof it. IPv6 grouped by /64	same, <code>pathpulse.service</code>
Memory bound	Rate-limit state in a TTL cache capped at 50,000 clients	same
Daily budgets	LLM 3,000, Geoapify 2,500, ElevenLabs 500 calls per day	<code>DailyBudget</code>
Caching	Explanations (LRU 4,096, keyed by evidence and model version), TTS audio (LRU 256), geocode results (1 h)	<code>services</code>
Single flight	Concurrent requests for one explanation share one LLM call	<code>ExplainService.explain</code>

Why `PUT /walks/{id}/position` stays on the general limit: at the expo every visitor shares the venue's NAT address, and each walker posts about 12 updates a minute. Counting those against the 30/min paid bucket would let three walkers lock out the whole hall. The protection moves to the resource instead: `RecentUpdates` rejects a second update for the same walk within 3 s (status changes excepted) before any Atlas read, and `POST /walks` (which creates documents) stays on the paid limit.

6. Transport and browser hardening

Headers set by Caddy on every response (verified live):

```

Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(self)
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com;
  font-src https://fonts.gstatic.com; img-src 'self' data: blob: https://tiles.openfreemap.org;
  connect-src 'self' https://tiles.openfreemap.org; worker-src 'self' blob;; child-src blob;; media-src
'self' blob;;
  frame-ancestors 'none'; base-uri 'self'; object-src 'none'
(Server header removed)

```

- `script-src 'self'`: no inline or third-party scripts. `'unsafe-inline'` is allowed for styles only (MapLibre and component styles).
- `worker-src blob:` and `child-src blob:` are needed by MapLibre's web workers; `media-src blob:` plays TTS audio.
- CORS: explicit origin list, methods GET, POST, PUT, header Content-Type, no credentials.
- The API binds to 127.0.0.1; ufw exposes only 22, 80, 443. The systemd unit runs as an unprivileged user with `ProtectSystem=strict`, `ProtectHome=true`, `ReadOnlyPaths=/srv/pathpulse`, `NoNewPrivileges`, `PrivateTmp`, an empty capability set, `LockPersonality`, and `RestrictSUIDSGID`.
- Errors never include stack traces (`unexpected_error_handler`).

7. LLM guardrails

The explanation chain is Groq → Gemini → Groq (smaller model) → deterministic template (`services/explain/`).

Guardrail	Implementation
No user text reaches the prompt	The user message is <code>Evidence (<kind>):\n<JSON></code> , built entirely from server data (<code>evidence.py</code>)
The LLM never produces a score	Scores, bands, and factor points come from the model; the prompt says "Every number you write must appear in it"
Grounding check	<code>validator.py</code> extracts every number (after removing evidence strings such as street names and "2020-2024") and every clock time; any number not in the evidence, or a time other than the evidence time label, rejects the text
Length and truncation	≤ 3 sentences, ≤ 420 characters; completions that did not finish normally are rejected
Banned framing	Case-insensitive word-boundary match on: safe, safest, safer, safety, guarantee, guaranteed, crime, criminal, dangerous area, dangerous neighborhood, bad neighborhood, bad area, sketchy, unsafe, violent, shooting, robbery, income, race, demographic
Prompt rules	Say "traffic risk", "lower-risk", "historical crashes"; never mention people, neighborhoods, or demographics; at most three factors and one action; ride modes say "riding"
Fallback	Any failure, timeout, or rejection moves to the next provider; the template is always valid and is not cached (so a transient outage does not stick)
Voice	<code>/tts</code> speaks only text the server produced for the same request
Community reports	Never included in evidence

UI copy is held to the same rules by tests: `frontend/src/components/safety/copy.test.tsx` renders every personal-safety surface and asserts none contains `safe`, `safest`, `unsafe`, `dangerous`, `bad area`, `sketchy`, or `guaranteed`; `frontend/e2e/demo.spec.ts` checks the rendered demo page ("copy never promises safety").

8. Secret handling and the key-masking incident

- **Where secrets live.** `backend/.env` only (gitignored; `.env.example` lists names). On the VM the file is streamed with `umask 077` and set to mode `0600`.
- **Never in the browser.** Geoapify, ElevenLabs, and LLM calls are proxied by the API.
- **Never in logs.** Settings use `SecretStr`; failures log `type(exc).__name__` only (exception text from database drivers can echo a connection string); `httpx` logging is raised to `WARNING` because Geoapify takes its key as a query parameter.
- **Operator tools.** `app.tools.check_keys` verifies each key with one live call and prints only `OK/MISSING/FAIL`. The key-capture helpers write matching strings straight to `backend/.env`.

Incident (commit 41dd776, Sat 2026-09-26 11:02 ET). The browser helper that saved newly created sponsor keys into `backend/.env` masked key-like strings in its own terminal output with a word-boundary regular expression. When the output was JSON, an escaped newline (`\n`) sat directly against a key, the word boundary did not match, and one key was printed unmasked in a local terminal session. It was never committed or deployed. Fix: JSON output is now decoded and each string value is masked individually, and non-JSON output has escaped newlines expanded before masking. The exposed key was flagged for revocation in `docs/sponsor_checklist.md` (the deploy uses a separate, newly created key). Lesson recorded: mask decoded values, not serialized text.

Scanning.

- `.pre-commit-config.yaml` runs **gitleaks** (v8.21.2) on every commit, plus `ruff`, `ruff-format`, `check-added-large-files` (5 MB), `check-merge-conflict`, and `end-of-file-fixer`. The hook is installed in this clone (`.git/hooks/pre-commit`).
- `gitleaks detect` over the full history (131 commits at the time of writing, run with `gitleaks 8.30.1`): **no leaks found**.
- `gitleaks dir .` over the working tree flags only `backend/.env`, the intended, gitignored secrets file.
- Dependency pinning: `uv.lock` (Python, installed with `--frozen` on the VM) and `package-lock.json` (installed with `npm ci`). No automated vulnerability scanner (for example `pip-audit` or `npm audit` in CI) is configured.

9. Fairness safeguards

Safeguard	Evidence
No demographic, income, race, or crime features in the traffic models	Feature list in <code>model/dataset.py</code> ; ARC income, race, and environmental-justice flags are never read (<code>network/features.ARC_STRUCTURAL</code>); the ARC baseline is labelled "demographic flags removed"
Crime never enters routing	<code>EdgeSignals</code> (the only safety input to the router) holds lighting and activity codes, no crime; <code>backend/tests/test_safety_router.py::test_crime_counts_never_change_the_route</code> builds bundles with crime counts $\times 0$, $\times 1$, $\times 1,000$ and asserts identical plans for both route preferences
Crime never enters explanations	Evidence builders read only traffic-model output; the validator also rejects crime vocabulary
No stigma from empty data	Crime bands use an exposure-normalized Empirical-Bayes posterior; a hex with zero reports can never be "higher" (<code>safety/banding.py</code> , tested in <code>data/tests/test_safety_crime.py</code>)
Unknown is not bad	Unknown lighting is never treated as unlit; unknown activity is neutral in routing
Always-on context	The fairness note appears wherever crime counts do

Narrow crime scope	Four offenses against persons, public places only, hex-aggregated
The lit-and-busy preference is opt-in and bounded	After dark only; at most 10% extra traffic exposure; must improve lighting/foot traffic by $\geq 15\%$ or the default route is kept
Honest framing	Scores describe where and when crashes concentrate, not a per-person probability; "traffic risk" and "lower-risk" wording; model card discloses exposure bias and reporting bias

10. Residual risks and follow-ups

- Access-log coordinates are not rounded (section 2).
- Rate limits and caches are per process; scaling past one worker needs a shared store.
- `/openapi.json` is public (no secrets in it, but it documents every endpoint).
- No automated dependency-vulnerability scanning.
- Follow links are bearer URLs: anyone the walker shares them with can see the walk until it ends or expires.
- The exposed key from the masking incident must be confirmed revoked in the provider console (listed as open in `docs/sponsor_checklist.md`).

Testing and quality

How PathPro is tested, what the suites cover, the numbers measured for this document, and the known gaps. All counts below were produced by running the suites on 2026-09-26 (Apple M1, macOS) against the working tree at commit `f090bd4`.

1. Strategy

- **Test first.** Features were built RED → GREEN: a `test:` commit with failing tests, then a `feat:` / `fix:` commit that makes them pass. Of 137 commits, 36 are `test:` commits (13 explicitly marked "(RED)"), 44 `feat:`, 19 `fix:`, 4 `perf:`, 2 `refactor:`, 26 `docs:`, 6 `chore:`.
- **Tests that guard product rules, not just code paths.** The most important tests encode promises made to users:

Rule	Test
Factor bars sum exactly to the displayed score	<code>backend/tests/test_scoring.py::test_attribution_always_sums_exactly</code> (Hypothesis, 200 examples); <code>test_api.py::test_segment_detail_factors_sum_to_score</code> ; <code>test_areas.py::test_area_lookup_factors_sum_to_score</code>
Exported log density equals the model's $EB \times multiplier / length$	<code>export/assemble._check_identity</code> (assertion inside every export, tolerance 1e-6); <code>data/tests/test_temporal.py::test_contributions_sum_to_log_multiplier</code> ; <code>data/tests/test_factors.py</code>
Routes never exceed the detour budget	<code>backend/tests/test_router.py::test_pathpro_route_respects_detour_budget</code> ; <code>test_safety_router.py::test_choose_lit_plan_respects_budget_and_exposure_cap</code> ; <code>test_modes.py::test Ride_router_uses_mode_speed_and_the_same_detour_rule</code>
Crime never changes a route	<code>backend/tests/test_safety_router.py::test_crime_counts_never_change_the_route</code> (crime $\times 0$, $\times 1$, $\times 1,000$)
A hex with no reports is never "higher"	<code>data/tests/test_safety_crime.py::test_posterior_bands_never_mark_a_hex_without_reports_higher</code>
LLM text with an unknown number, a promise, or crime framing is rejected	<code>backend/tests/test_explain.py</code>
Explanations fall back to the template within the time budget	<code>test_explain.py::test_timeouts_and_errors_fall_back_to_template_within_budget</code>
UI copy never uses banned framings	<code>frontend/src/components/safety/copy.test.tsx</code> ; <code>frontend/e2e/demo.spec.ts</code> "copy never promises safety"
The ride export never changes a walk byte	<code>data/tests/test_ride_export.py</code>
Share-my-walk tokens never appear in responses; only the owner can update	<code>backend/tests/test_walks_api.py</code> , <code>test_walks_service.py</code> , <code>test_walks_repo.py</code>
Deduplication and snapping match golden cases	<code>data/tests/test_dedupe.py</code> , <code>test_snap.py</code>
The offline demo works with the network off	<code>frontend/e2e/demo.spec.ts</code> (<code>context.setOffline(true)</code>)

- **Fakes, not live services.** Backend tests use a bundle factory (`backend/tests/bundle_factory.py`, `safety_factory.py`) that writes a small synthetic bundle, an in-memory Mongo collection (`fake_mongo.py`), `respx` for HTTP providers, and a fixed clock. No test calls a paid API; a `network` marker exists for live checks and is not used by default.
- **Frontend.** Vitest + React Testing Library + `vitest-axe` in `jsdom`; `msw` for HTTP; behaviour-focused queries by role and name. WebGL components (`src/map/**`) and the app shell are excluded from unit coverage and exercised by Playwright.

2. Suites and measured results

Suite	Command	Result	Time
Data pipeline	<code>uv run --package pathpulse-data pytest data/tests -q -p no:cacheprovider</code>	222 passed (27 test files)	42 s
Backend API	<code>uv run --package pathpulse-backend pytest backend/tests -q -p no:cacheprovider</code>	277 passed , 1 warning (Starlette <code>TestClient</code> deprecation notice) (26 test files)	29 s
Frontend unit and component	<code>cd frontend && npx vitest run --coverage</code>	541 passed in 66 test files	79 s
End-to-end (Playwright)	<code>cd frontend && npx playwright test</code>	19 tests defined: desktop project 16 (<code>live.spec.ts</code> 8, <code>demo.spec.ts</code> 5, <code>gps.spec.ts</code> 3), phone project (Pixel 7) 3 (<code>mobile.spec.ts</code>). Not re-run for this document (it starts <code>uvicorn</code> and <code>Vite</code> and renders WebGL on <code>SwiftShader</code>).	n/a
Total automated tests		1,040 unit/integration tests passing + 19 e2e tests	

Playwright configuration: `timeout` 90 s, `expect` 20 s, `SwiftShader` GL flags, `retries`: 1 on CI only, traces kept on failure, web servers started for the API (`/healthz`) and `Vite`.

3. Coverage

Package	Measured	Threshold	Notes
Backend (app)	91.6% lines (<code>--cov=app --cov-config=backend/pyproject.toml</code>)	80% (fail under)	Passes
Frontend (src, excluding map/Web GL shell)	98.8% lines, 97.4% statements, 92.5% branches, 96.8% functions	80 / 80 / 75 / 80	Passes

Data (pathpulse_data, package config)	78.9% lines (<code>--cov=pathpulse_data --cov-config=data/pyproject.toml</code>)	80% (fail under)	Below threshold. <code>citywide/export.py</code> , <code>citywide/features.py</code> , <code>citywide/model.py</code> (City Pulse) are at 0% because they are exercised only by running the pipeline; <code>export/assemble.py</code> 49%, <code>network/graph.py</code> 48%, <code>network/bike.py</code> 67%, <code>network/coverage.py</code> 63%, <code>model/dataset.py</code> 77%. <code>data/pyproject.toml</code> already omits other IO-only CLIs from coverage; the City Pulse modules are not in that omit list.
---------------------------------------	---	------------------	--

Note on the documented command: `uv run --package pathpulse-data pytest data/tests --cov` from the repository root picks up no coverage configuration (the root `pyproject.toml` has none), so it measures test files as well and reports 89%. The package-scoped numbers above are the meaningful ones.

4. Static analysis

Tool	Command	Result
ruff (lint: E, F, W, I, N, UP, B, SIM, RUF, S, PTH; line length 100)	<code>uv run ruff check data backend</code>	All checks passed
mypy (strict = true)	<code>uv run mypy data/src backend/app</code>	47 errors in 26 files (112 files checked)
TypeScript	<code>cd frontend && npx tsc -b --noEmit</code>	Passes (exit 0)
oxlint	<code>cd frontend && npx oxlint</code>	Warnings only (for example <code>react(set-state-in-effect)</code> in <code>hooks/useTypewriter.ts</code> , <code>react(only-export-components)</code> in <code>components/Timeline.tsx</code>); no errors
gitleaks	pre-commit hook; <code>gitleaks detect</code> on history	No leaks in history (see security_privacy.md §8)

Known mypy errors (pre-existing)

By category: 14 `arg-type`, 13 `no-any-return`, 6 `call-overload`, 5 `index`, 4 `dict-item`, and one each of `unused-ignore`, `type-arg`, `return-value`, `no-untyped-call`, `assignment`. Most are in the data pipeline around pandas and NumPy typing (for example `export/writers.py`, `model/temporal_data.py`, `db/load_tiger.py`). Backend examples:

- `app/main.py:53–58`: `GroqProvider/GeminiProvider` are frozen dataclasses whose `name` is read-only, while the `Provider` protocol declares `name` as a settable attribute. Declaring `name` as a read-only property in the protocol would fix it.
- `app/services/geocode.py`: the `params` dict is inferred as `dict[str, object]`.
- `app/tools/record_demo.py`: mixed-type dict literals.

None of these affects runtime behaviour (the test suites pass), but the "mypy clean" goal in the build plan is not met.

5. Reviews

Independent reviewer agents were run at milestones, and each review's findings became their own `fix:` commits:

Review	Agent	Resulting commit
--------	-------	------------------

Model evaluation honesty (spatial-block bootstrap, grouped CV folds, fair temporal baseline, HIN at its own length share)	<code>ecc:mle-reviewer</code>	<code>a2d87d9</code>
Security (rate limits, budgets, worker thread for routing, error envelopes)	<code>ecc:security-reviewer</code>	<code>554600d</code>
Mobile UX redesign (route-metre progress, preview arrival, named GPS starts, status screen)	code review of the redesign	<code>26010e4</code>
Personal-safety extension (optional export guard, StreetLight error, in-memory walk throttle, day-part literal)	code and security review	<code>0a046c3</code>
Deploy scripts (system Python under <code>ProtectHome</code> , <code>sslip.io</code> HTTPS before DNS, surfacing failures)	found while going live, not an agent review	<code>47fc47e</code>

Workflow per milestone (from `CLAUDE.md`): plan → failing test (commit) → implement (commit) → reviewers → verification loop (ruff, mypy, tsc, lint, tests with coverage, gitleaks) → checkpoint.

6. Latency sample

Measured for this document, in-process with FastAPI's `TestClient` on an Apple M1 laptop, real bundle `pp-20260926-1902-f49c0d2`, no API keys and no database (so no Atlas report lookup), `cond: "dry"`, depart Friday 22:30. Random origins around Georgia Tech; successful plans only.

Mode	Trips	p50	p95	Max
Walk (0.8–3 km)	40	47 ms	78 ms	246 ms
Bike (2–8 km)	40	40 ms	85 ms	135 ms

Bundle load at startup: 0.6 s. This excludes network time and the production VM's slower single vCPU, so treat it as a lower bound; the PRD target is $p95 < 1.5$ s end to end (NFR-03). Earlier measurements recorded in the repository: route planning p95 fell from 1.95 s to 0.26 s after the citywide router refactor (`c8d5e8f`), and default plus lit-and-busy plans together ran p50 154 ms, p95 283 ms on 40 random night trips (`docs/safety_sources.md`).

7. Known issues and gaps

Item	Status
Data package coverage 78.9% vs the 80% gate (City Pulse modules untested)	Open
47 mypy strict errors	Open, runtime-neutral
Playwright suite not re-run for this document	19 tests exist; run <code>npx playwright test</code> from <code>frontend/</code> before a release
No CI pipeline in the repository (<code>.github/</code> absent); checks run locally and in the pre-commit hook	Open
No automated dependency-vulnerability scan	Open
Temporal and ride models have no bootstrap intervals on their deviance numbers; City Pulse capture has no interval	Documented in data_and_models.md
Starlette <code>TestClient</code> deprecation warning	Cosmetic